

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING IN POLYMER INFORMATICS

Day 4

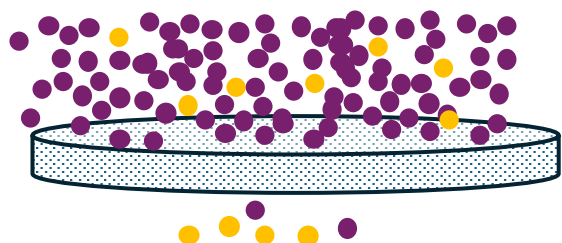
Generative Models and Reinforcement Learning

Generative Models and Reinforcement Learning

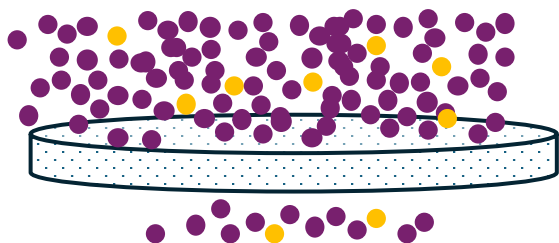
Candidate library (Beads)

Promising candidates (Golden beads)

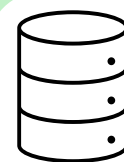
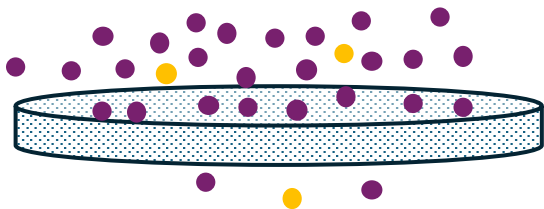
ML model (Filter)



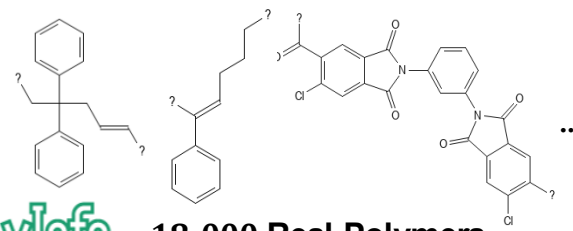
Poor effect ML model



Low quality candidate library

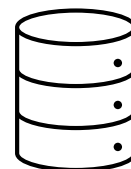


Real Polymer Structures

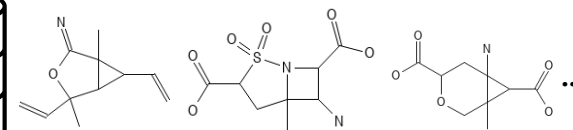


PolyInfo

~18,000 Real Polymers



Small Molecule Compounds



PubChem

~116M Compounds

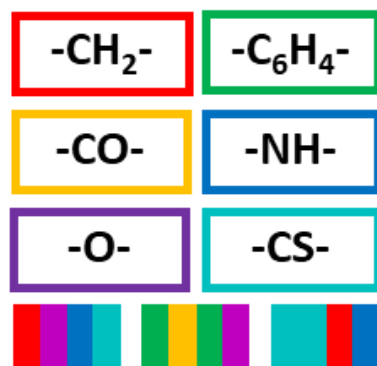
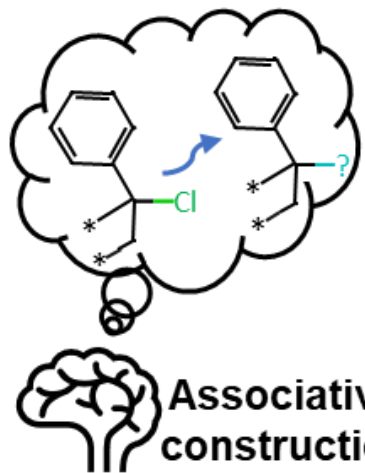
GDB-13

~977M Compounds

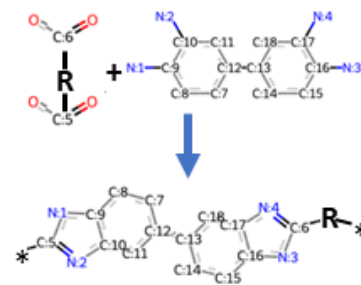
GDB-17

~50M Compounds

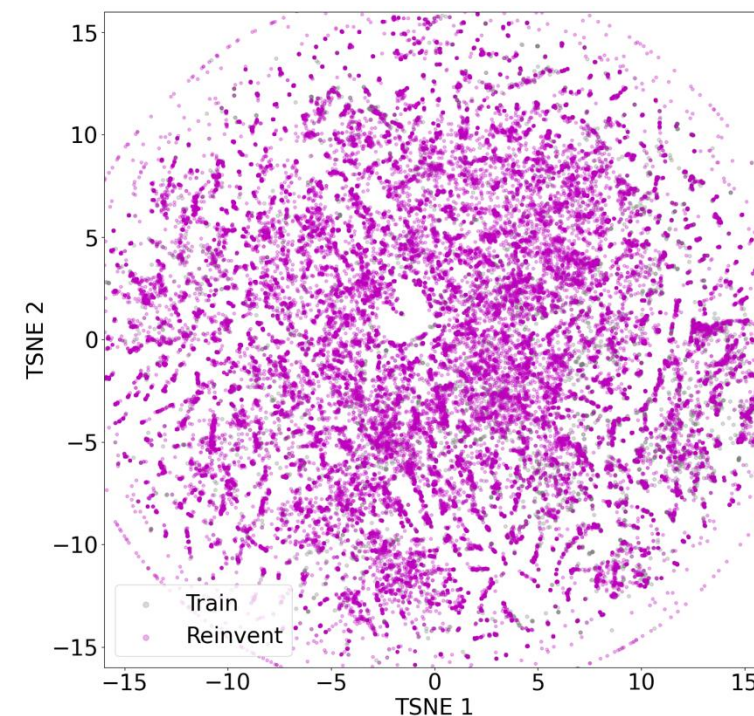
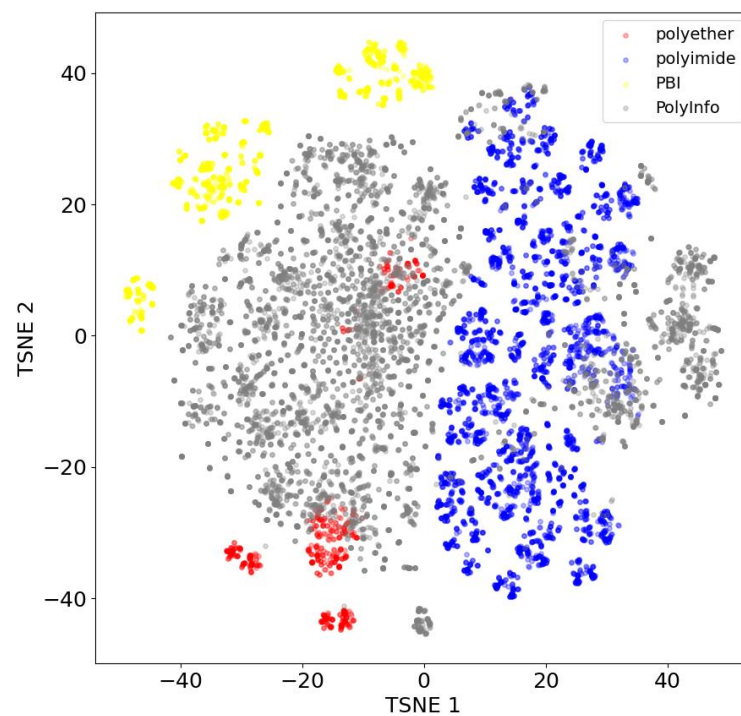
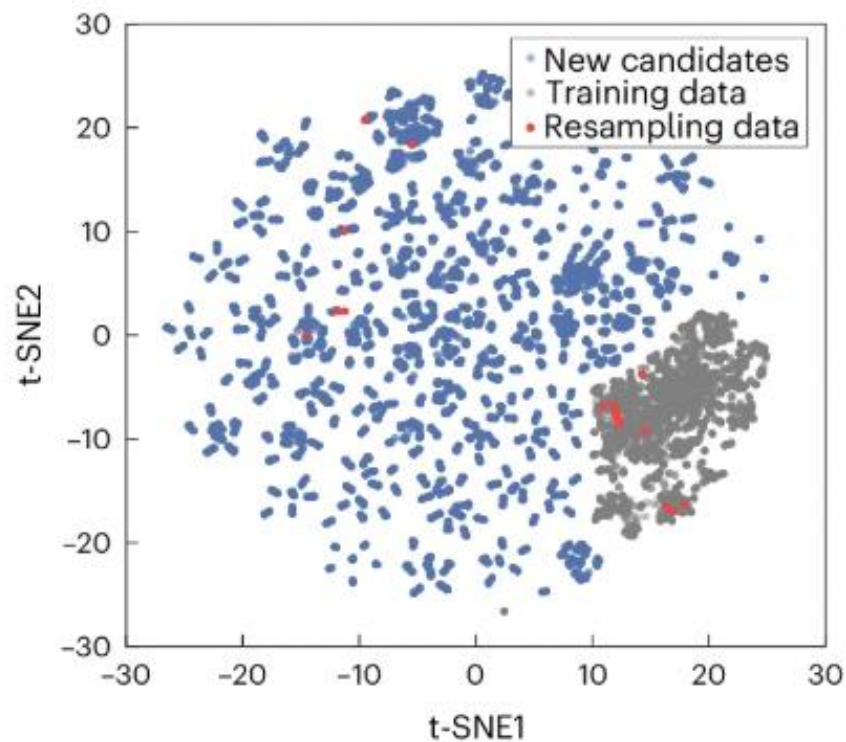
Generative Models and Reinforcement Learning



Building blocks assembly



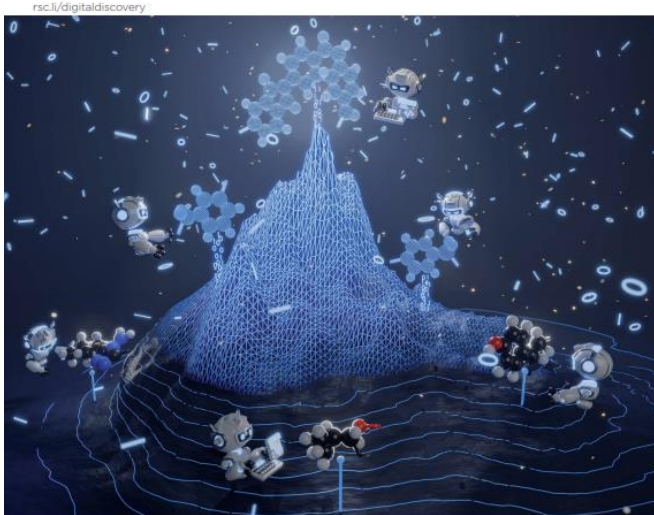
Generative models



Generative Models and Reinforcement Learning

Digital
Discovery

Volume 4
Number 4
April 2025
Pages 883-1114



ISSN 2635-098X

ROYAL SOCIETY
OF CHEMISTRY

PAPER
Ying Li et al.
Benchmarking study of deep generative models for inverse
polymer design

Digital
Discovery

ROYAL SOCIETY
OF CHEMISTRY

PAPER

[View Article Online](#)

[View Journal](#) | [View Issue](#)



Cite this: *Digital Discovery*, 2025, 4,
910

Received 16th December 2024
Accepted 27th January 2025

DOI: 10.1039/d4dd00395k

rsc.li/digitaldiscovery

Benchmarking study of deep generative models for inverse polymer design†

Tianle Yue,^a Lei Tao,^b Vikas Varshney^c and Ying Li^{d,*}

Molecular generative models based on deep learning have increasingly gained attention for their ability in *de novo* polymer design. However, there remains a knowledge gap in the thorough evaluation of these models. This benchmark study explores *de novo* polymer design using six popular deep generative models: Variational Autoencoder (VAE), Adversarial Autoencoder (AAE), Objective-Reinforced Generative Adversarial Networks (ORGAN), Character-level Recurrent Neural Network (CharRNN), REINVENT, and GraphINVENT. Various metrics highlighted the excellent performance of CharRNN, REINVENT, and GraphINVENT, particularly when applied to the real polymer dataset, while VAE and AAE show more advantages in generating hypothetical polymers. The CharRNN, REINVENT, and GraphINVENT models were successfully further trained on real polymers using reinforcement learning methods, targeting the generation of hypothetical high-temperature polymers for extreme environments. The findings of this study provide critical insights into the capabilities and limitations of each generative model, offering valuable guidance for future endeavors in polymer design and discovery.

8 January 2025. Downloaded on 9/3/2025 8:56:26 PM.
under a Creative Commons Attribution-NonCommercial 3.0 Unported Licence.

Generative Models and Reinforcement Learning

Code: <https://github.com/yt10410/Polymer-Generative-Models-Benchmark>

 **Polymer-Generative-Models-Benchmark** Public

 Pin

 Unwatch 1

 main ▾

 1 Branch

 1 Tag

 yt10410 Update README.md

 MOSES

 REINVENT&GraphINVENT

 Reinforcement_Learning

 Well-trained models

 README.md

Download the Moses Docker image.

```
docker pull molecular sets/moses
```



Create a container.

```
nvidia-docker run -it --name moses --network="host" --shm-size 10G molecularsets/moses
```



Copy files from the local machine to the Moses Docker container.

```
docker cp /Your address/PolyInfo_train.csv [CONTAINER ID]:/moses/data/PolyInfo_train.csv
docker cp /Your address/PolyInfo_test.csv [CONTAINER ID]:/moses/data/PolyInfo_test.csv
docker cp /Your address/train_polymer.py [CONTAINER ID]:/moses/scripts/train_polymer.py
```



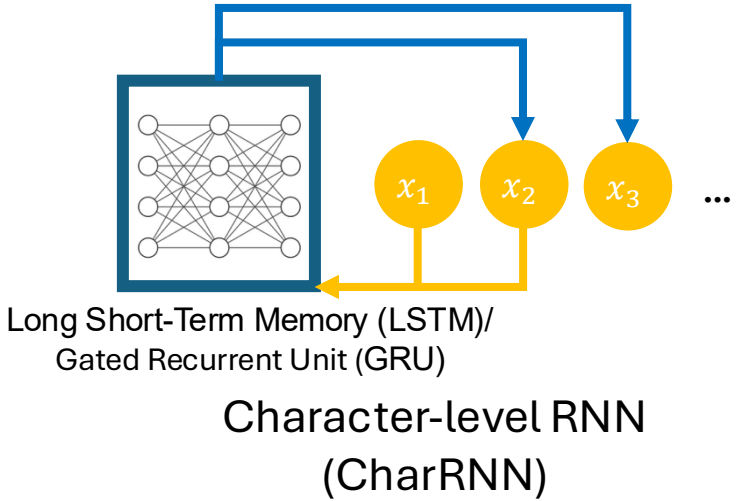
The PolyInfo dataset can be found here (`PolyInfo_train.csv` and `PolyInfo_test.csv`), the PubChem and GDB-13 datasets can be downloaded from: <https://zenodo.org/records/13821449>.

Load the Moses Docker container.

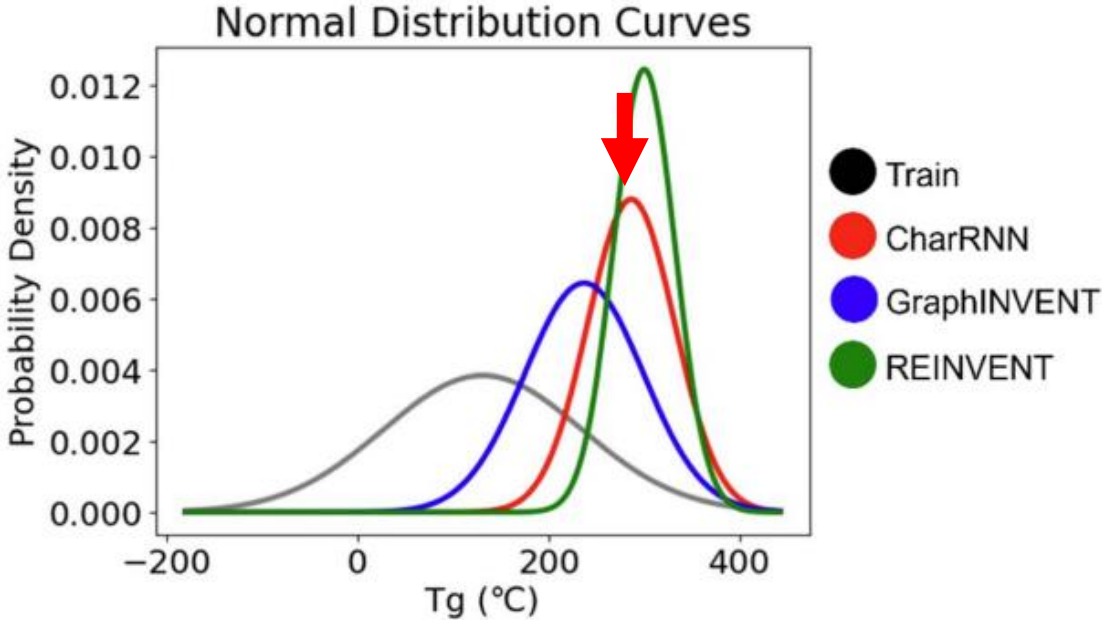
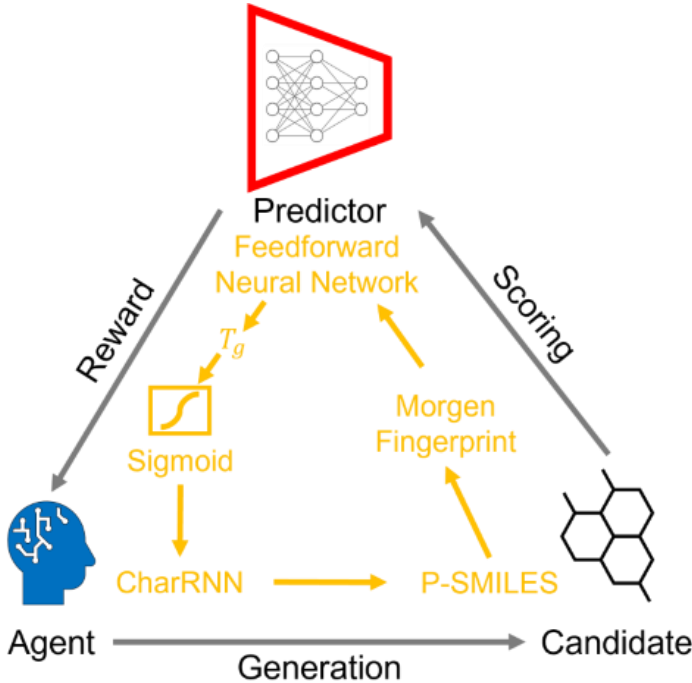
```
docker start moses
docker attach moses
```



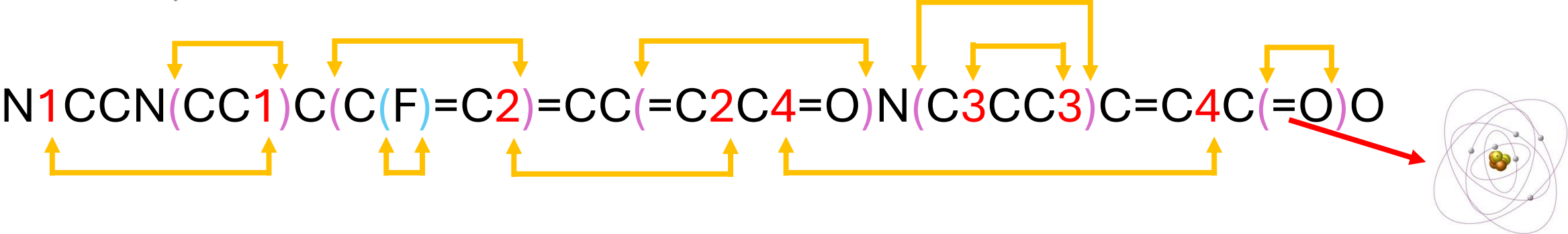
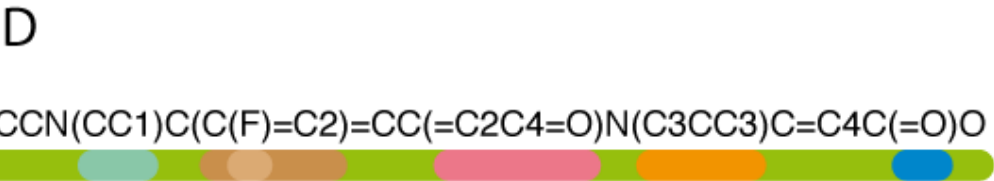
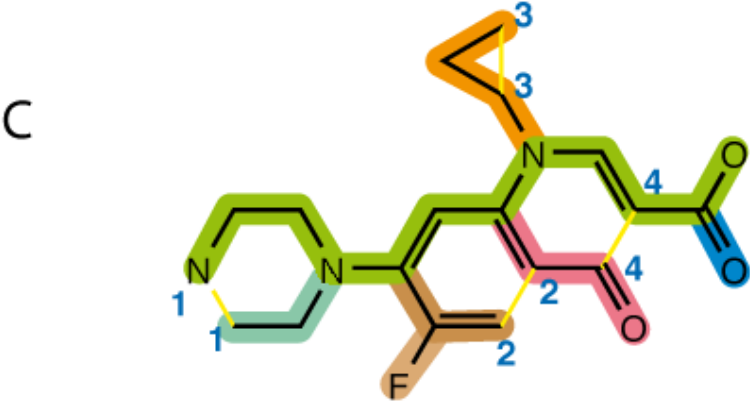
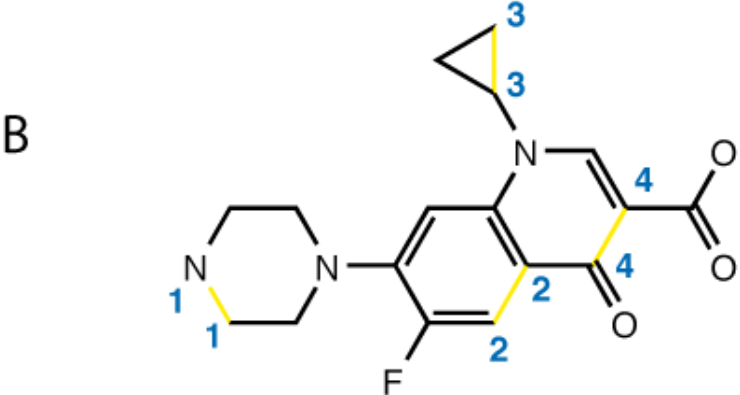
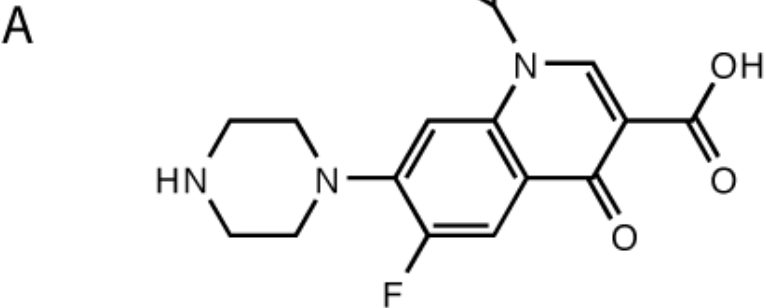
Generative Models and Reinforcement Learning



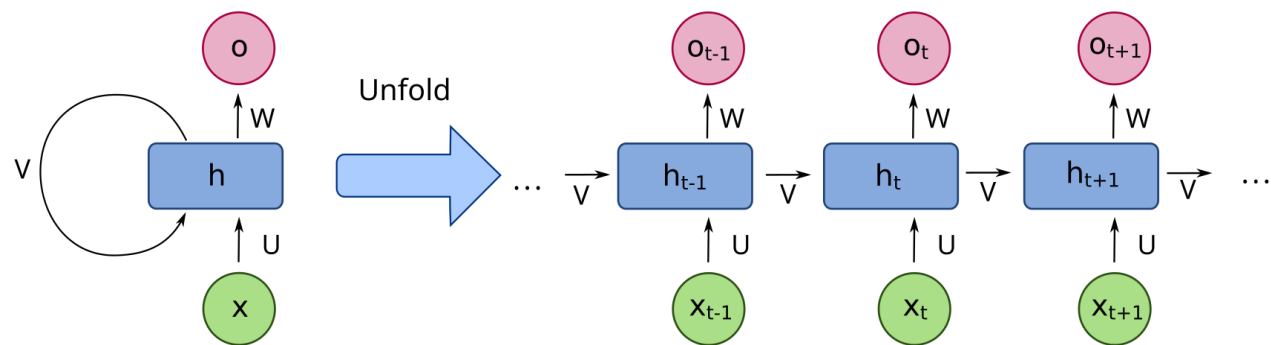
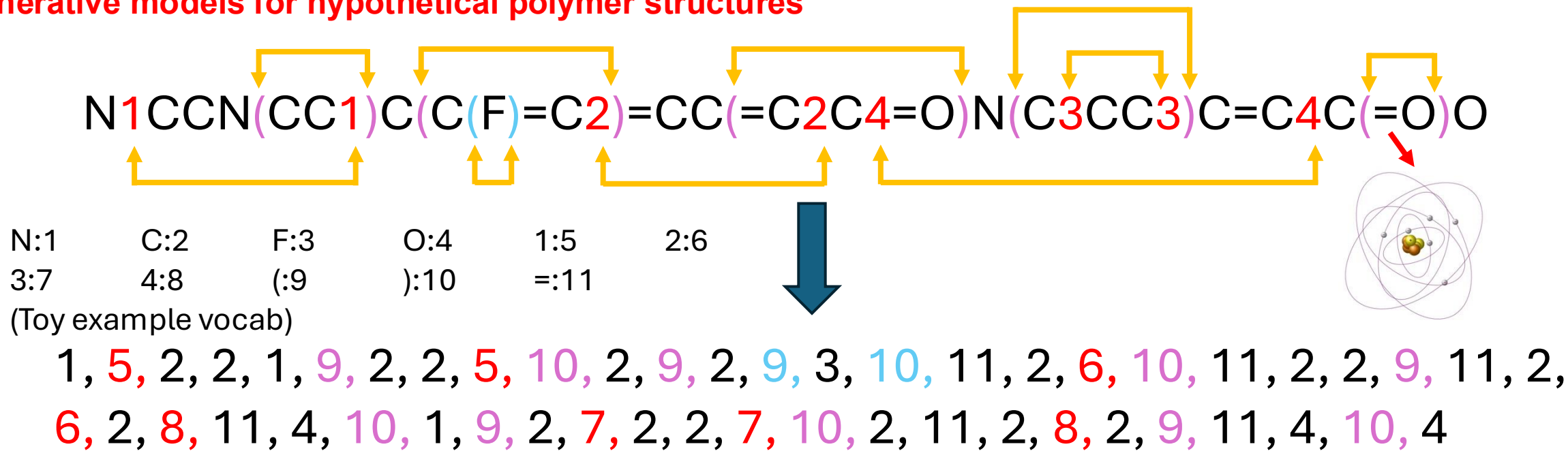
Models	Low data volume	High atomic count	Complex types of atoms
AAE	★	★★	★★
CharRNN	★★★	★★★	★★
GraphINVENT	★★★	★	★
ORGAN	★	★	★
REINVENT	★★	★★★	★★★
VAE	★	★★	★★



Generative models for hypothetical polymer structures



Generative models for hypothetical polymer structures



Recurrent neural network (RNN)



Cite this: *Mater. Horiz.*, 2026, 13, 5961

Received 28th January 2026,
Accepted 14th April 2026

DOI: 10.1039/d6mh00159a

rsc.li/materials-horizons

LadderGen: a large-scale generative library of ladder polymers for membrane separations

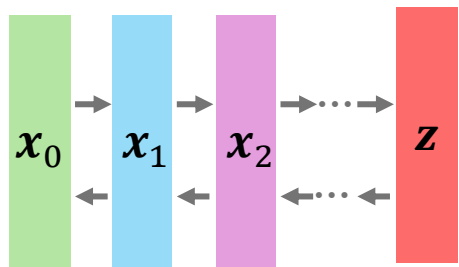
Tianle Yue,^{†a} Subhamoy Mahajan,^{†a} Nathaniel Straight^b and Ying Li  ^{*a}

Ladder polymers possess exceptional rigidity, microporosity, and thermal stability, yet their exploration as gas separation membranes is hindered by the scarcity of synthetically accessible chemical structures. Here, we introduce LadderGen, a large-scale hypothetical ladder polymer library constructed using template polymerization reactions and advanced generative machine learning (ML) models, expanding the chemical space of ladder polymers to 0.8 million chemical structures. Existing ML models were used to predict key membrane properties, such as glass transition temperature, fractional free volume, and O₂/N₂ ideal selectivity and permeability, enabling rapid screening of the generated library to identify promising candidate ladder polymers that may approach or exceed the 2008 Robeson upper bound. Finally, high-fidelity molecular dynamics simulations provide validation of the predicted gas permeabilities of the top candidates. This work provides a framework for accelerated *de novo* ladder polymer discovery to identify promising next-generation gas-separation membranes for sustainable energy and environmental applications.

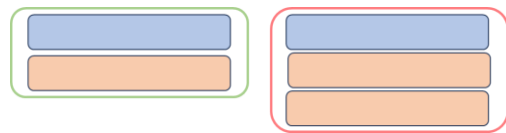
New concepts

Ladder polymers are promising membrane materials but remain severely underexplored due to limited synthetically accessible structures. Here we demonstrate a synthesis-aware, generative discovery paradigm for ladder polymers by introducing LadderGen, a large-scale digital library (~0.8 million) built in two complementary ways: (i) template polymerization reactions that explicitly bridge readily available small-molecule precursors to ladder repeat units (maintaining synthetic plausibility), and (ii) multiple generative ML models that expand this chemically valid ladder space beyond what reaction templates alone can enumerate. This differentiates LadderGen from prior polymer-screening efforts that either overlook ladder architectures or cannot scale because ladder-polymer data are scarce. The broader insight is a general, multi-fidelity materials-discovery workflow—synthesis-aware generation → rapid ML property screening → targeted high-fidelity MD verification—that makes inverse design feasible for an underexplored polymer class and enables efficient identification of ladder candidates with competitive permeability–selectivity trade-offs for energy-relevant separations.

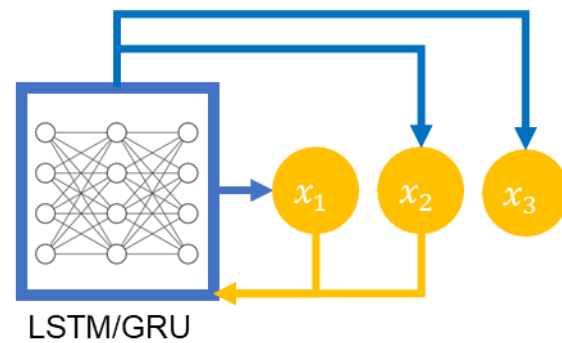
Generative Models



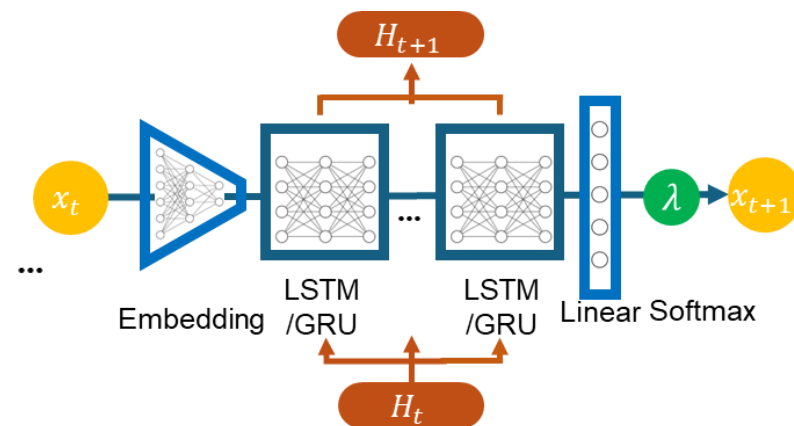
Diffusion



minGPT

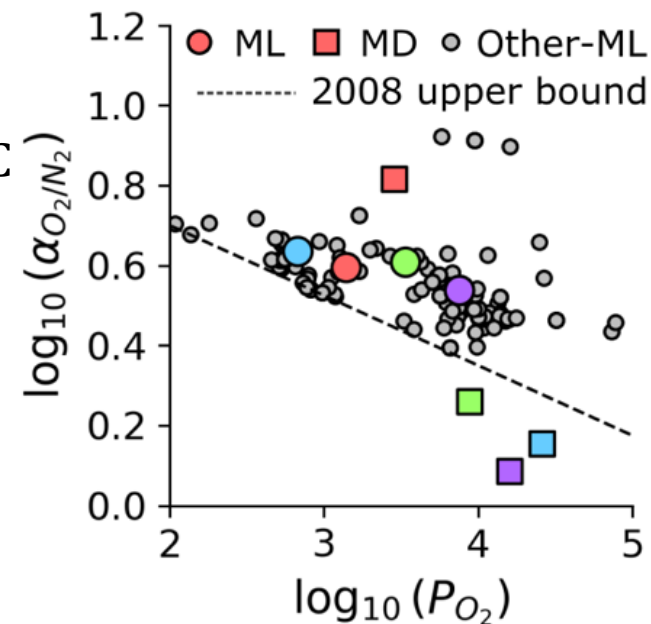
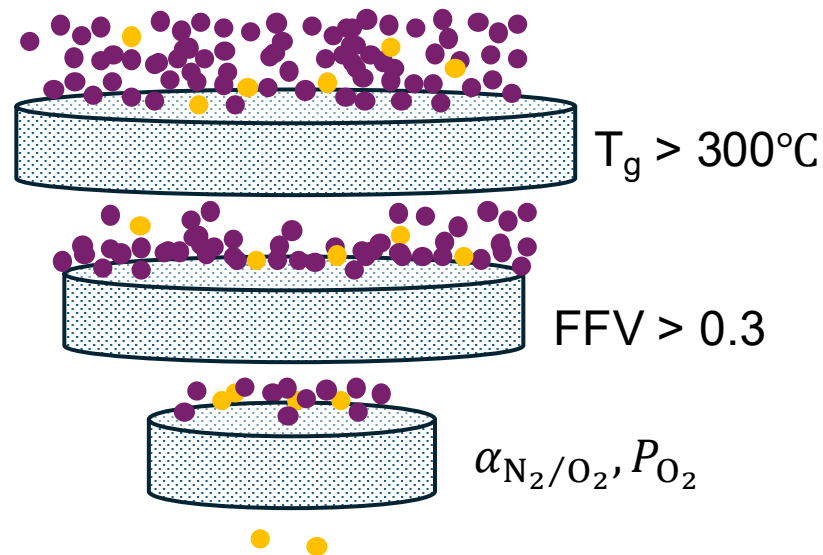


CharRNN



REINVENT

Library	Count
Real Ladder Polymer	140
NAS	30,925
Tröger's Base	42,269
REINVENT (finetune)	2,469
REINVENT	144,666
CharRNN	79,944
Diffusion	142,570
minGPT	579,225
Total	799,842



Generative Models

Code: <https://github.com/ytl0410/LadderGen>



LadderGen

Public

main 1 Branch 0 Tags Go to file

yt10410 Create ladder_polymer_filter.py

LICENSE Initial commit

README.md Update README.md

diffusion1D.ipynb Add files via upload

ladder_polymer_filter.py Create ladder_polymer_filter.py

minGPT.ipynb Add files via upload

valid_polymer_smiles.py Create valid_polymer_smiles.py

README MIT license

LadderGen

Well-trained models and generative outcomes for the paper "LadderGen: A Large-Scale Generative Library of Ladder Polymers for Membrane Separation."

Generation results

All generation results can be found at: <https://zenodo.org/records/18275495>

Well-trained models

Well-trained CharRNN, REINVENT, minGPT, and Diffusion models can be found at: <https://zenodo.org/records/18275968>

CharRNN

Download the Moses Docker image.

```
docker pull molecular_sets/moses
```

Create a container.

```
nvidia-docker run -it --name moses --network="host" --shm-size 10G molecularsets/moses
```

Request GPU

CO

4_Generative_models.ipynb

☆

☁

FileEditViewInsertRuntimeToolsHelp

Commands

UndoCtrl+M Z

RedoCtrl+Shift+Y

Select all cellsCtrl+Shift+A

Cut cell or selection

Copy cell or selection

Paste

Delete selected cellsCtrl+M D

Find and replaceCtrl+H

Find nextCtrl+G

Find previousCtrl+Shift+G

Notebook settings

Clear all outputs

Framework for material candidates

Candidates

PI

In the prompt, the utilization of ML techniques for generating candidates. However, this method requires complex structures for research. These structures is notably

Notebook settings

Runtime type

Python 3

Hardware accelerator

☐ CPU

☒ T4 GPU

☐ A100 GPU

☐ L4 GPU

☐ v5e-1 TPU

☐ v2-8 TPU

☐ v6e-1 TPU

Want access to premium GPUs? [Purchase additional compute units](#)

Runtime version

Latest (recommended)

☐ Hide generative AI features

☐ Automatically run the first cell or section on any execution

☐ Omit code cell output when saving this notebook

Cancel

Save

Set up Python environment

```
!pip install rdkit
!pip install torch==2.2.0
!pip install torchaudio==2.2.0 torchvision==0.17.0
!pip install pytorch_lightning==1.9.0
!pip install torchmetrics==0.5.0
!pip install selfies
!pip install numpy==1.26.4
```

Install RDKit package

Install a specific version of PyTorch package

A package for small-molecule structure representation, included only to satisfy certain package dependencies here.

Install a specific version of numpy package

Restart session

WARNING: The following packages were previously imported in this runtime:

[numpy]

You must restart the runtime in order to use newly installed versions.

Restarting will lose all runtime state, including local variables.

CancelRestart session

[1]
✓ Os

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount(

Here, modify the path where you saved this notebook and the associated lab files on your Google

[2]
✓ Os

```
path = '/content/drive/MyDrive/SPE/'
```

Check the path name is assigned correctly.

[3]
✓ Os

```
!echo $path
import os
os.path.isdir(path)
```

/content/drive/MyDrive/SPE/

True

[4]
✓ Os

```
# This is needed so can import packages from a different path than standard libraries
import sys
sys.path.append(path)
```

Links your Google drive to the Colab session after restart session

Train/Pre-train a CharRNN

```
import os

import torch
import pytorch_lightning as pl
from pytorch_lightning.callbacks import EarlyStopping, ModelCheckpoint
from pytorch_lightning.loggers import CSVLogger

import rdkit.Chem as Chem
import selfies as sf
import pandas as pd
import numpy as np

from lstm_climber import SELFIESDataModule, SMILESDataModule, LanguageModel
from lstm_climber.datamodules import SMILESDataSet
import lstm_climber.utils as utils
```

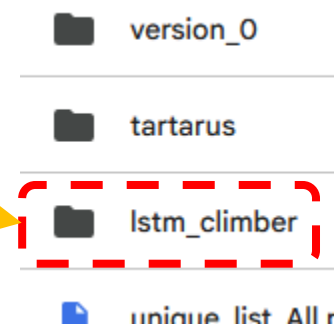
Deep learning framework

Cheminformatics

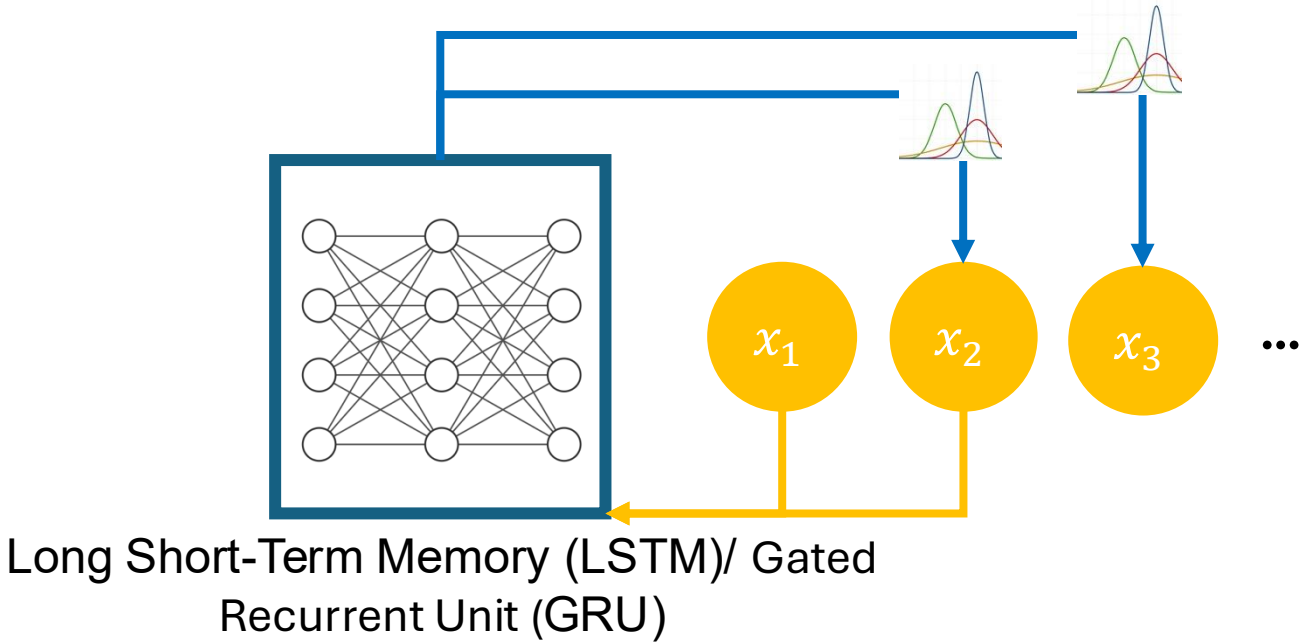
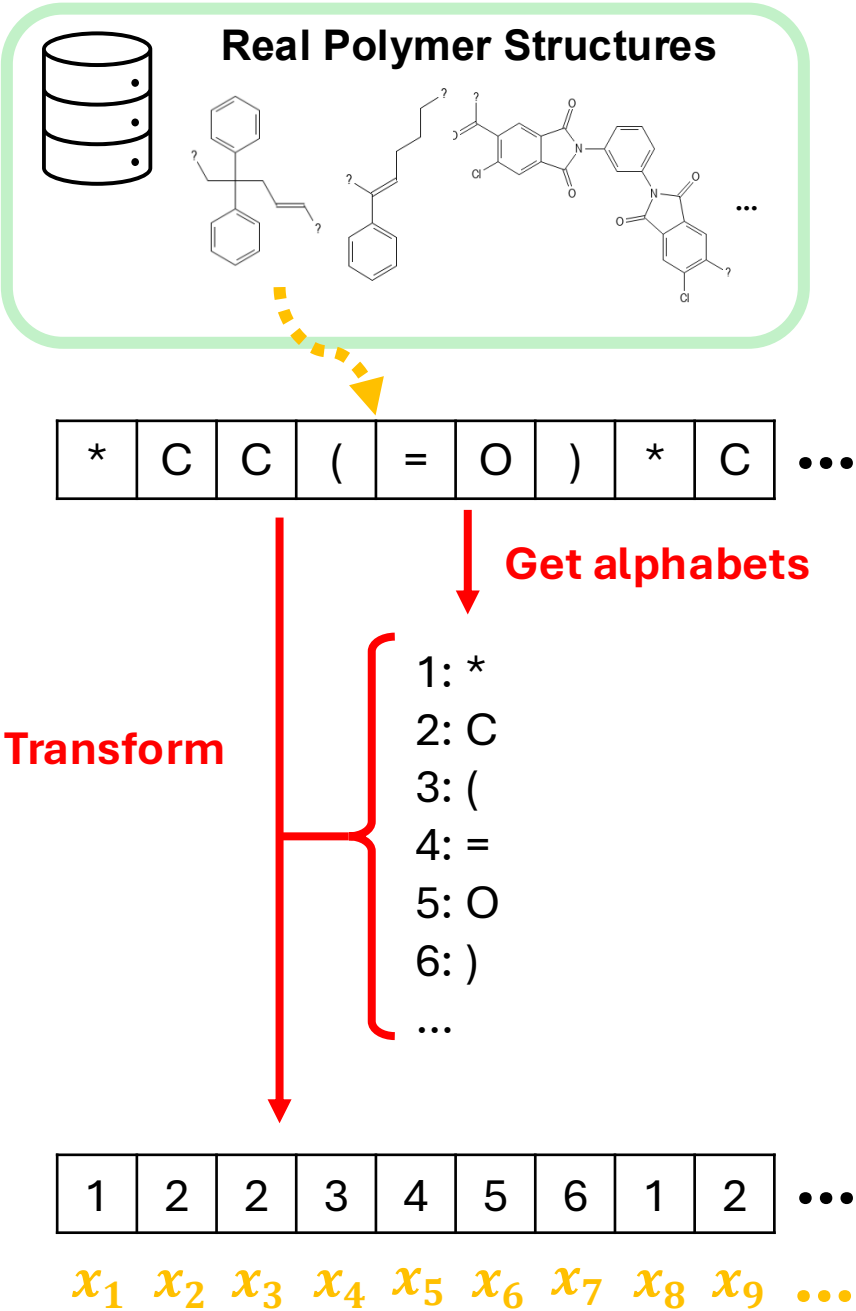
Data management

Scientific computing package

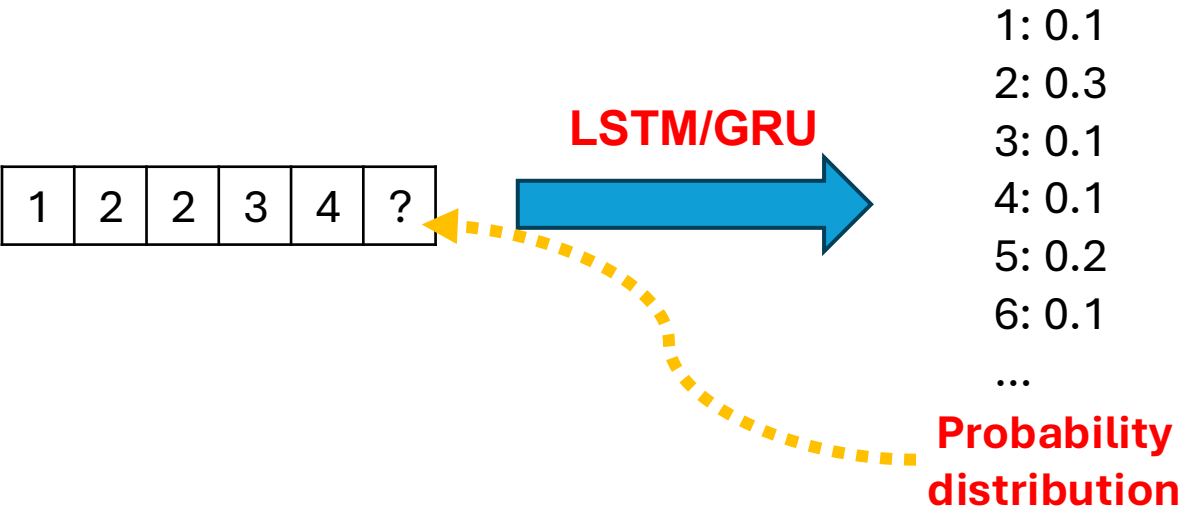
Functions from Tartarus packages



Train/Pre-train a CharRNN



Output of LSTM/GRU: A probability distribution covering all characters in the alphabet



Train/Pre-train a CharRNN

```
# define parameters
```

```
data_path      = os.path.join(path, 'PolyInfo.csv') → Set the training dataset path
```

```
string_type    = 'smiles' # 'smiles'
```

```
num_workers    = 6
```

SMILES is used to describe molecular structures

```
# get the data
```

```
#smi_list, sfs_list = utils.get_lists(data_path, sep='\n', header='infer', smiles_name='smiles')
```

```
smi_list = pd.read_csv(data_path)['smiles'] → Read in the training dataset
```

```
if string_type == 'selfies':
```

```
    str_list = sfs_list
```

```
    dm = SELFIESDataModule(str_list, batch_size = 128, num_workers = num_workers)
```

```
elif string_type == 'smiles':
```

```
    str_list = smi_list
```

```
    dm = SMILESDataModule(str_list, batch_size = 128, num_workers = num_workers) → Transform the polymer SMILES data into a data module
```

```
else:
```

```
    raise ValueError('No such string representation.')
```

```
# print some stuff about the data
```

```
print(f'You are using the "{string_type}" representation.')
```

```
print(f'Number of molecules: {len(str_list)}')
```

```
print(f'Length of longest molecule: {dm.len_molecule}')
```

```
print(f'Length of alphabet: {dm.len_alphabet}')
```

```
print(f'Alphabet: {dm.alphabet}')
```

You are using the "smiles" representation.

Number of molecules: 12841

Length of longest molecule: 306

Length of alphabet: 49

Alphabet: ['#', '%', '(', ')', '*', '+', '-', '/', '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '=', 'A', 'B', 'C', 'F', 'G', 'H', 'I', 'K',

Summarize dataset information

All alphabets involved in the dataset

Train/Pre-train a CharRNN

Hidden layer size

Number of layers

✓
13m

```
[9] # create model
model = LanguageModel(1024, 3, dm.len_alphabet, dm.len_molecule)

# default logger used by trainer
logger = CSVLogger(os.path.join(os.getcwd(), 'trained_models'), name=string_type)

callbacks = [
    ModelCheckpoint(dirpath = logger.log_dir, filename = 'final_model', monitor = 'val_loss', mode = 'min', verbose=True),
    EarlyStopping(monitor='val_loss', patience=20, mode='min', verbose=True)
]

trainer = pl.Trainer(
    accelerator = 'gpu',
    devices=1,
    logger=logger,
    max_epochs = 5,
    callbacks = callbacks,
)
trainer.fit(model, dm)
print('Finished training!')

metrics = utils.plot_metrics(os.path.join(logger.log_dir, 'metrics.csv'), logger.log_dir)
print('Done!')
```

Log file save directory

The logging tool of PyTorch

Automatically saves the best model when the validation loss decreases

Training ends early to save compute resources if the validation loss doesn't decrease within 20 epochs

Training epochs. The number of epochs is set to 5 for demonstration purposes. In typical training scenarios, this parameter is usually set between 70-100, which requires significantly more computation time to complete.

Plot of the recorded metrics

Train/Pre-train a CharRNN

```
WARNING:lightning_fabric.loggers.csv_logs:Missing logger folder: /content/trained_models/smiles
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.utilities.rank_zero:IPU available: False, using: 0 IPU
INFO:pytorch_lightning.utilities.rank_zero:HPU available: False, using: 0 HPU
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
INFO:pytorch_lightning.callbacks.model_summary:
| Name          | Type          | Params
-----
0 | lstm          | LSTM          | 21.2 M
1 | linear        | Linear        | 50.2 K
2 | loss_fn       | CrossEntropyLoss | 0
3 | train_vae_acc | Accuracy      | 0
4 | val_vae_acc   | Accuracy      | 0
-----
21.2 M   Trainable params
0        Non-trainable params
21.2 M   Total params
84.988   Total estimated model params size (MB)
INFO:pytorch_lightning.callbacks.early_stopping:Metric val_loss improved. New best score: 0.659
INFO:pytorch_lightning.utilities.rank_zero:Epoch 0, global step 81: 'val_loss' reached 0.65866 (best 0.65866), saving model to '/content/trained_models/smiles/version_0/final_model.ckpt' as top 1
Epoch: 0      validation loss: 0.6544244885444641

INFO:pytorch_lightning.callbacks.early_stopping:Metric val_loss improved by 0.034 >= min_delta = 0.0. New best score: 0.625
INFO:pytorch_lightning.utilities.rank_zero:Epoch 1, global step 162: 'val_loss' reached 0.62467 (best 0.62467), saving model to '/content/trained_models/smiles/version_0/final_model.ckpt' as top 1
Epoch: 1      validation loss: 0.620803952217102

INFO:pytorch_lightning.callbacks.early_stopping:Metric val_loss improved by 0.013 >= min_delta = 0.0. New best score: 0.612
INFO:pytorch_lightning.utilities.rank_zero:Epoch 2, global step 243: 'val_loss' reached 0.61159 (best 0.61159), saving model to '/content/trained_models/smiles/version_0/final_model.ckpt' as top 1
Epoch: 2      validation loss: 0.6077431440353394

INFO:pytorch_lightning.callbacks.early_stopping:Metric val_loss improved by 0.030 >= min_delta = 0.0. New best score: 0.581
INFO:pytorch_lightning.utilities.rank_zero:Epoch 3, global step 324: 'val_loss' reached 0.58134 (best 0.58134), saving model to '/content/trained_models/smiles/version_0/final_model.ckpt' as top 1
Epoch: 3      validation loss: 0.5804761648178101

INFO:pytorch_lightning.callbacks.early_stopping:Metric val_loss improved by 0.003 >= min_delta = 0.0. New best score: 0.579
INFO:pytorch_lightning.utilities.rank_zero:Epoch 4, global step 405: 'val_loss' reached 0.57881 (best 0.57881), saving model to '/content/trained_models/smiles/version_0/final_model.ckpt' as top 1
Epoch: 4      validation loss: 0.577442467212677
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped: `max_epochs=5` reached.
Finished training!
Metrics plotted!
Done!
```

GPU is identified

GPU device ID

Model information

Training logs

Generate hypothetical polymer structures using the trained CharRNN

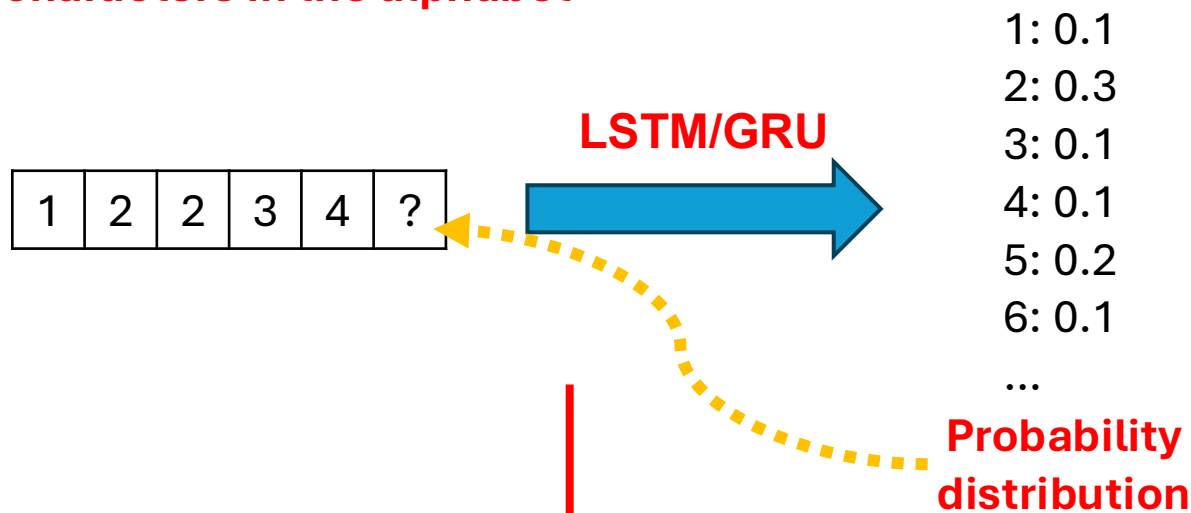
```
# define parameters
model_path      = path+ '/version_0/final_model.ckpt'
# control sampling
num_generations = 100
temperature     = 1.1
```

Path of the model

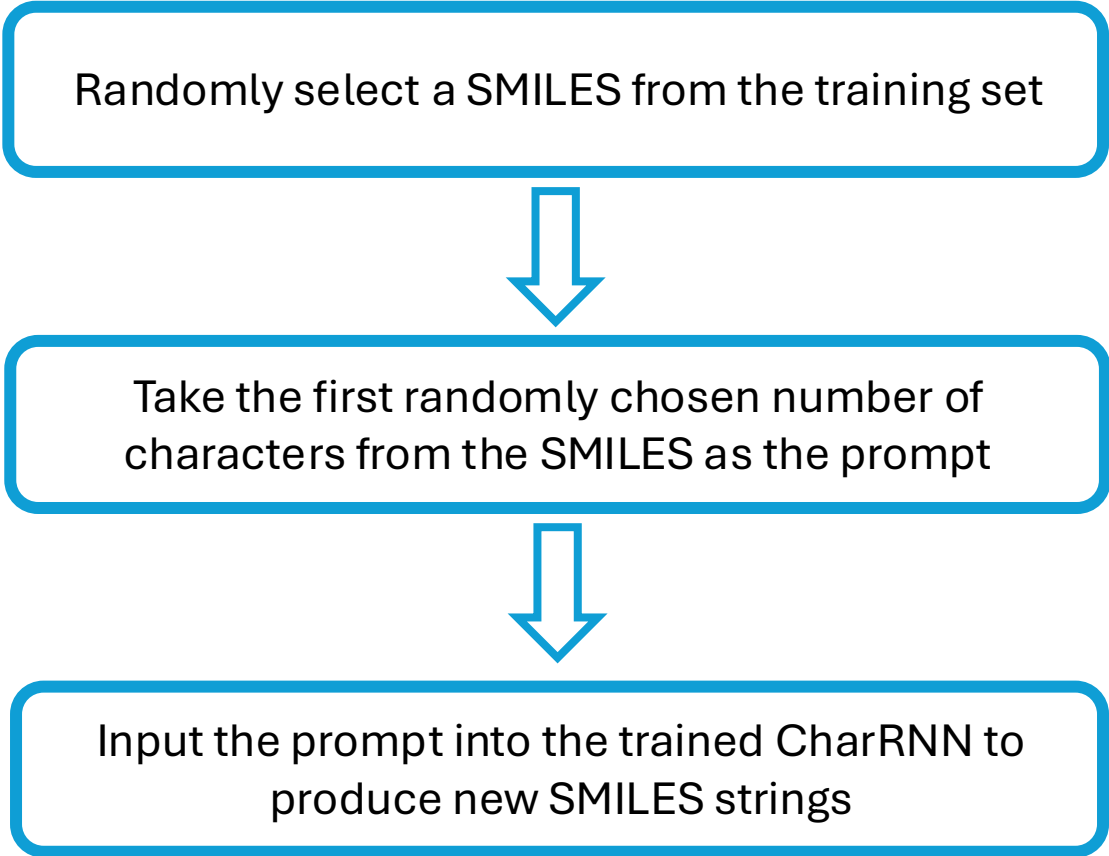
Number of generated structures

Control the randomness of the generation process
< 1.0 is less random, > 1.0 is more random

Output of LSTM/GRU: A probability distribution covering all characters in the alphabet



When generating new results with CharRNN, an initial prompt must be provided.



Generate hypothetical polymer structures using the trained CharRNN

```
# define parameters
```

```
model_path = path+ '/version_0/final_model.ckpt' → Path of the model
```

```
# control sampling
```

```
num_generations = 100 → Number of generated structures
```

```
temperature = 1.1 # < 1.0 is less random, > 1.0 is more random → Control the randomness of the generation process
```

```
model = LanguageModel.load_from_checkpoint(model_path) → Load the model from the setting path
rand_string = pd.DataFrame({'smiles': str_list}).sample(frac=1).reset_index(drop=True) → Randomly shuffle SMILES
num_samps = num_generations
new_df = pd.DataFrame(columns=['smiles']) → Create a new DataFrame to store the generated SMILES
for i, s in enumerate(rand_string['smiles']):
    if num_samps > 0:
        frac = np.random.rand()/2.0 + 0.25 # from 0.25 to 0.75
        len_fn = sf.len_selfies if string_type == 'selfies' else len
        seed = utils.get_n_char(s, int(len_fn(s) * frac), string_type)
        onehot_seed = dm.encode_string(seed)
        sampled_molecules = model.sample(onehot_seed, 1, temperature = temperature) → Extract the initial portion of the SMILES
        if string_type == 'selfies':
            new_smiles, new_selfies = dm.logits_to_smiles(sampled_molecules, return_selfies=True)
            new = pd.DataFrame({'smiles': new_smiles, 'selfies': new_selfies})
        else:
            new_smiles = dm.logits_to_smiles(sampled_molecules, canonicalize=False) → Sample based on the initial portion
            new = pd.DataFrame({'smiles': new_smiles})
        new_df = pd.concat([new_df, new]) → Convert the generated results into SMILES
        num_samps = num_generations - len(new_df)
    else:
        break
        Append the generated results to the DataFrame
```

Generate hypothetical polymer structures using the trained CharRNN

11m

```
# define parameters
model_path = pat

# control sampling
num_generations = 100
temperature = 1.1

model = LanguageModel
rand_string = pd.D
num_samps = num_ge
new_df = pd.DataFr
for i, s in enumer
    if num_samps >
        frac = np.
        len_fn = s
        seed = uti
        onehot_see
        sampled_mc
        if string_
            new_sm
            new =
        else:
            new_sm
            new =
        new_df = p
        num_samps
    else:
        break
```

[23] new_df



smiles



0	*C(=O)c1ccc(C(=O)c2ccc(C=C3CCC(=Cc4ccc(*)o4)C3...
0	*C(=O)c1ccc2c(c1)C(=O)N(c1ccc3c(c1)C(=O)c1cc(N...
0	*Oc1ccc(CC(NC(=O)Cc2ccc(OC(=O)CCCCCCC(*)=O)cc2...
0	*CC(*)C(=O)OCc1ccccc1 ...
0	*CC(*)C(=O)NCCC(F)(F)C(F)(F)C(F)(F)C(F)(F)C(F)...
...	...
0	*c1ccc(Oc2ccc(-n3c(=O)c4c(Oc5ccc(OCCCC)cc5)c5c...
0	*OC(C)CCOC(=O)c1ccc(C(*)=O)cc1 ...
0	*c1ccc(C(C#N)(c2cccc(C(C#N)(c3ccccc3)c3ccc(Oc4...
0	*C(=O)c1ccc(C(=O)N(*)c2ccccc2)c(Oc2ccccc2)c1 ...
0	*c1ccc(OCCOc2ccc(C=Cc3ccc([N+](=O)[O-])cc3)c(O...

100 rows × 1 columns

generation process

file SMILES

ILES

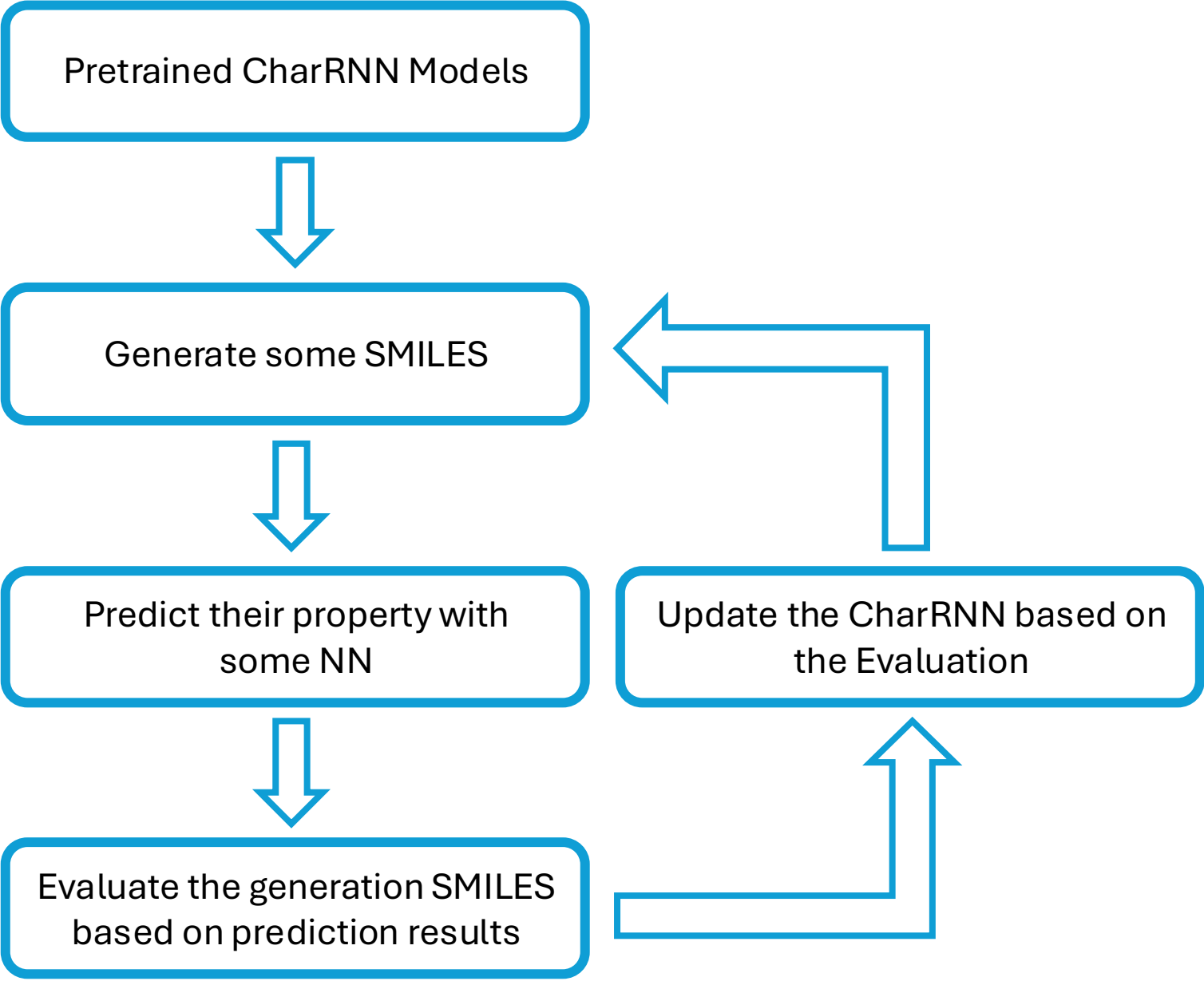
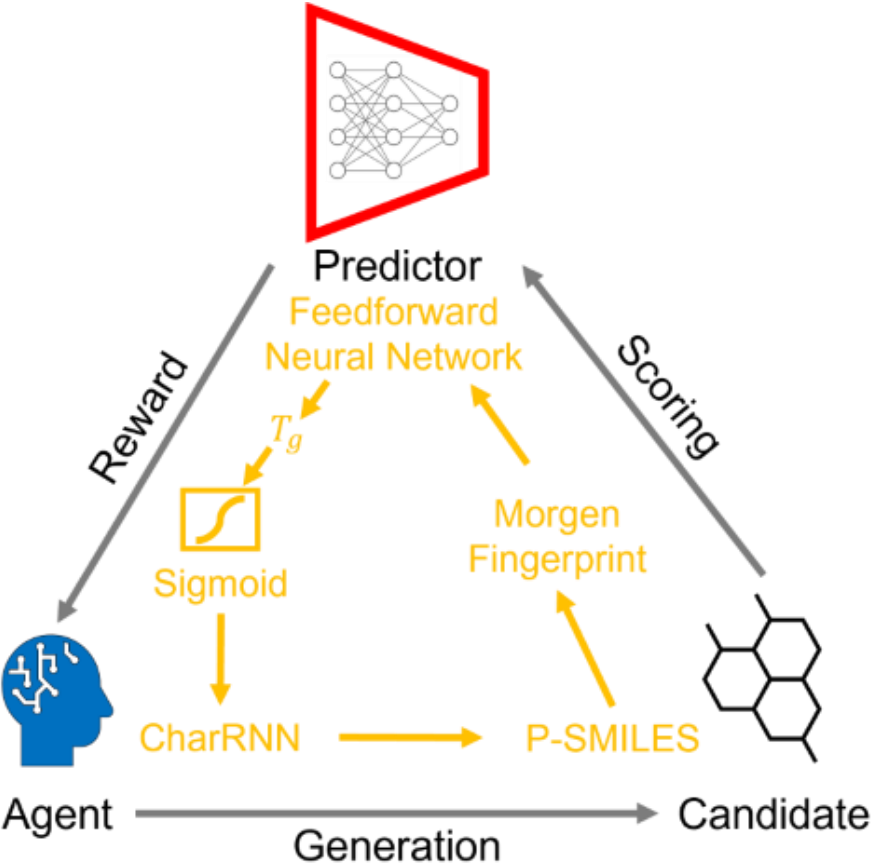
f the SMILES

the initial portion

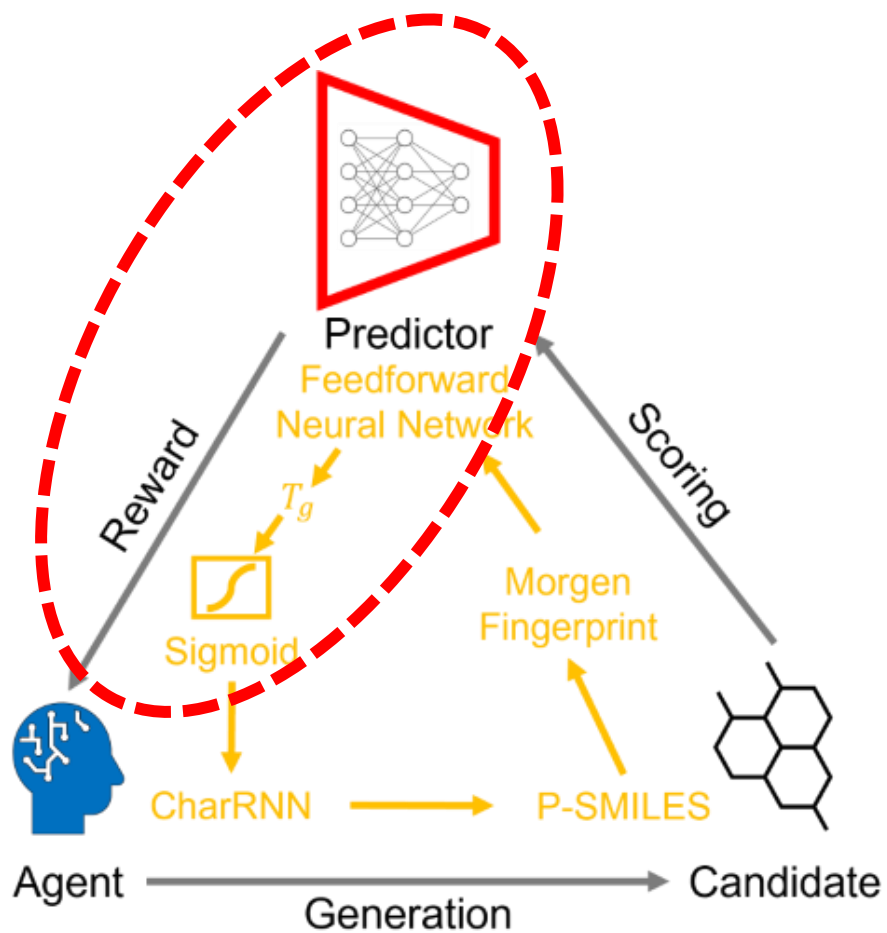
ated results into
SMILES

ie

Reinforcement learning



Reinforcement learning

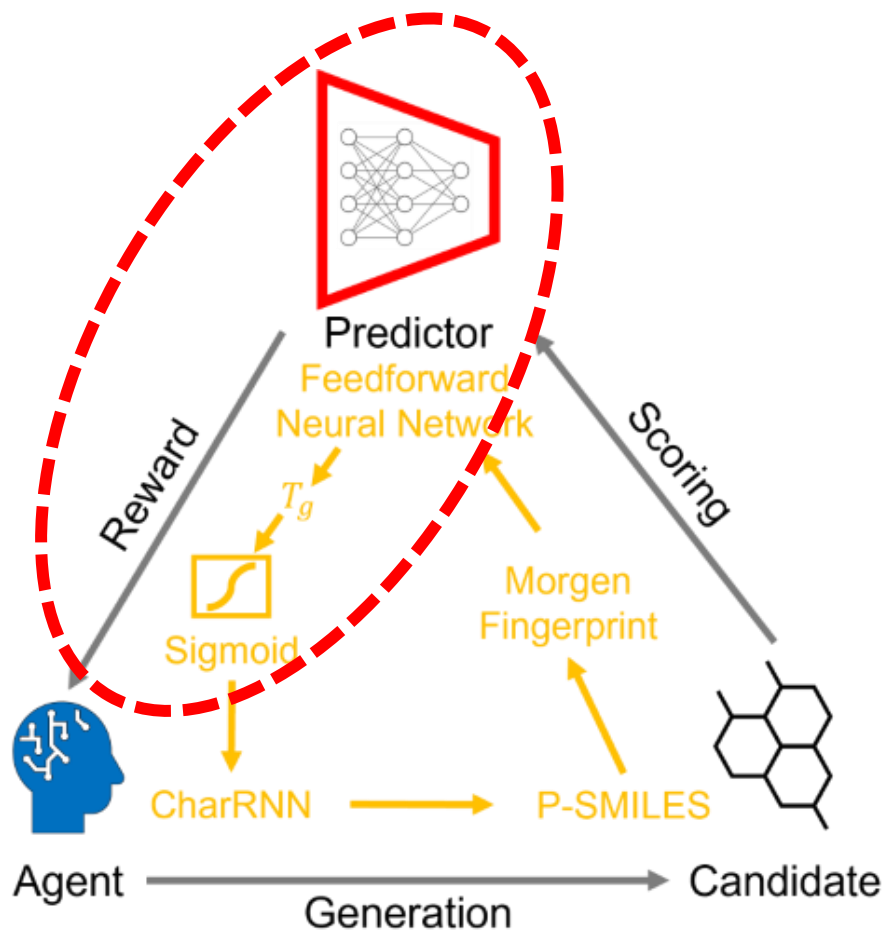


```
def fitness_function(smi: str):  
    try:  
        mol = Chem.MolFromSmiles(smi)  
        fn = AllChem.GetMorganFingerprintAsBitVect(mol, radius=3, nBits=1024)  
    except:  
        return 0  
    loaded_model = FFNN()  
    loaded_model.load_state_dict(torch.load(path+'Tg.pth'))  
    loaded_model.eval()  
  
    new_inputs = torch.tensor(fn, dtype=torch.float32)  
  
    with torch.no_grad():  
        predictions = loaded_model(new_inputs)  
    value = predictions.item()  
    score = 1 / (1 + math.exp(-0.014391156900419547 * (value - 200)))  
    return value
```

Load a neural network to predict the T_g .

Calculate the score based on the prediction results

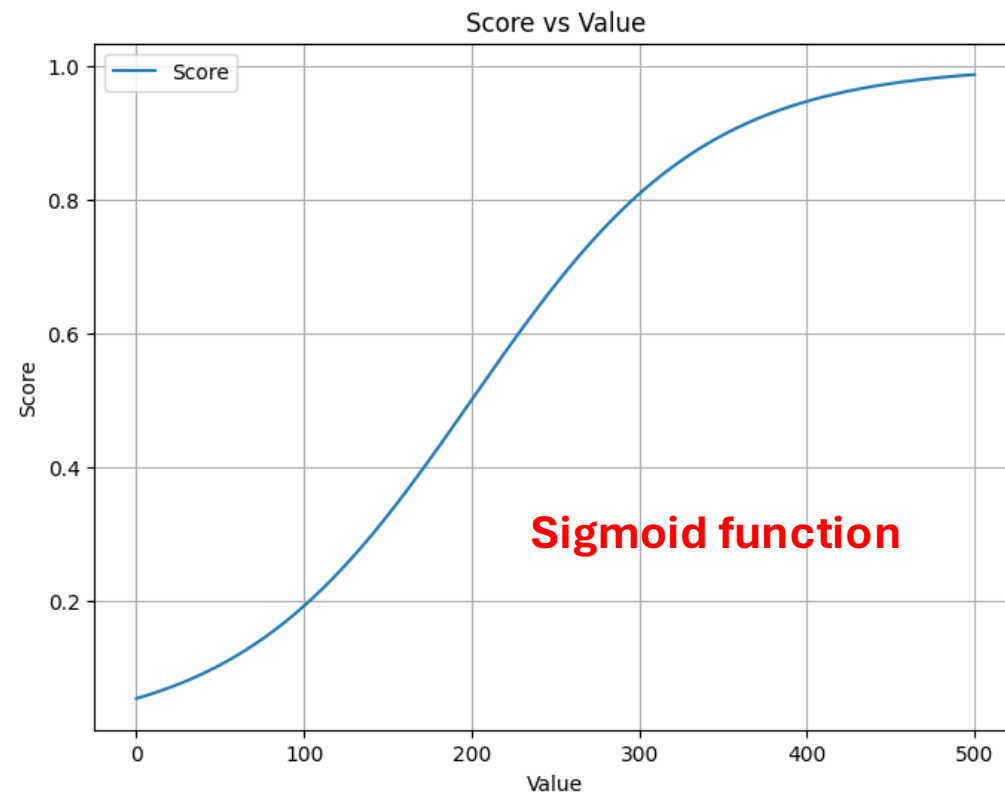
Reinforcement learning



```
import math
import matplotlib.pyplot as plt
import numpy as np
# Define the function
def compute_score(value):
    return 1 / (1 + math.exp(-0.014391156900419547 * (value - 200)))

# Generate a range of values
values = np.linspace(0, 500, 1000) # Adjust the range as needed
scores = [compute_score(value) for value in values]

# Plot the results
plt.figure(figsize=(8, 6))
plt.plot(values, scores, label='Score')
plt.xlabel('Value')
plt.ylabel('Score')
plt.title('Score vs Value')
plt.grid(True)
plt.legend()
plt.show()
```



Generate hypothetical polymer structures using the trained CharRNN

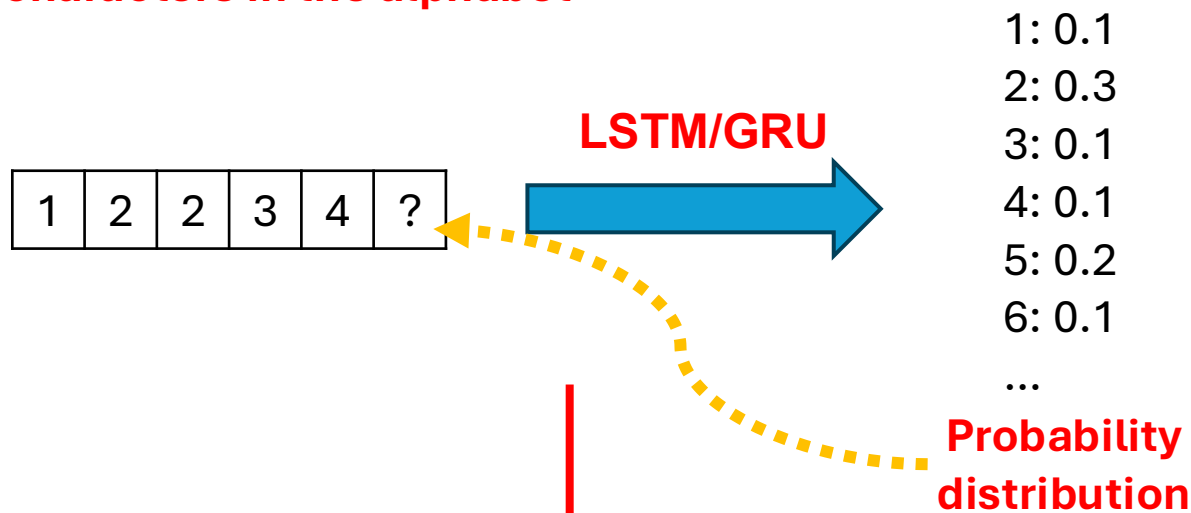
```
# define parameters
model_path      = path+ '/version_0/final_model.ckpt'
# control sampling
num_generations = 100
temperature     = 1.1
```

Path of the model

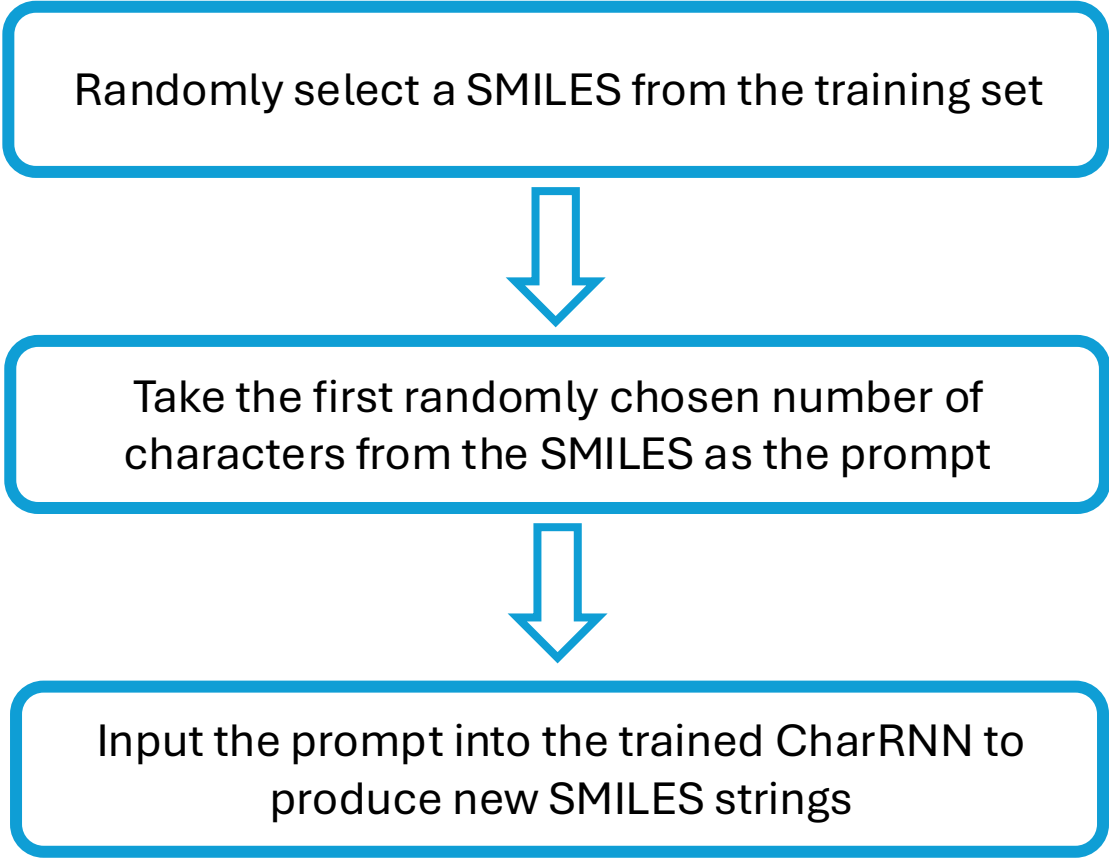
Number of generated structures

Control the randomness of the generation process
< 1.0 is less random, > 1.0 is more random

Output of LSTM/GRU: A probability distribution covering all characters in the alphabet



When generating new results with CharRNN, an initial prompt must be provided.



Generate hypothetical polymer structures using the trained CharRNN

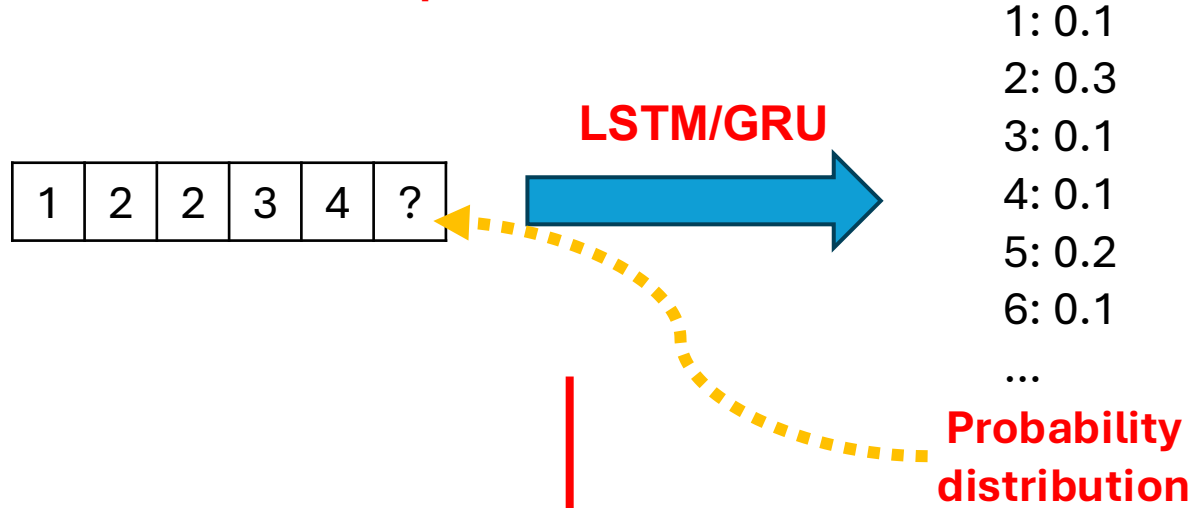
```
# define parameters
model_path      = path+ '/version_0/final_model.ckpt'
# control sampling
num_generations = 100
temperature     = 1.1
```

Path of the model

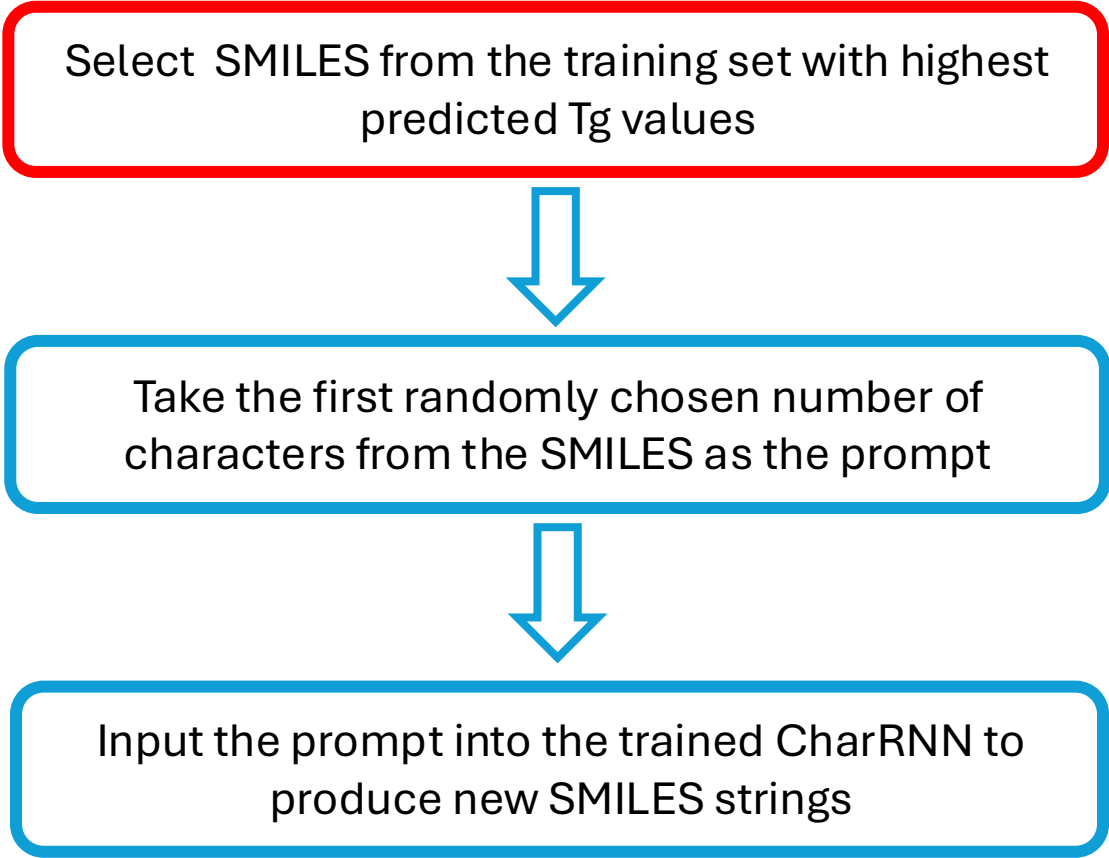
Number of generated structures

Control the randomness of the generation process
< 1.0 is less random, > 1.0 is more random

Output of LSTM/GRU: A probability distribution covering all characters in the alphabet



When generating new results with CharRNN, an initial prompt must be provided.



Reinforcement learning

```
# define parameters
```

```
model_path = path+'/version_0/final_model.ckpt'
```

→ Path of the pretrained model

```
out_path = 'RESULTS_retrain_{string_type}_PI'
```

→ Path for output

```
num_workers = 6
```

```
# control sampling
```

```
# total number of samples = num_best * samps_per_seed * num_randomize
```

```
num_generations = 15
```

→ Total training generations

```
num_best = 10
```

```
# number of top molecules to generate seeds
```

```
num_randomize = 5
```

```
# number of randomized seeds
```

```
samps_per_seed = 2
```

```
# number of samples for each seed
```

```
num_seed_chars = None
```

```
# number of initial characters to sample from
```

```
# if none, sample characters 1/4 - 3/4 of strings
```

```
temperature = 1.1
```

```
# < 1.0 is less random, > 1.0 is more random
```

```
retrain = True
```

```
# retrain the network on new molecules
```

→ Number of SMILES with the highest predicted Tg values

```
print(f'Total numbers searched per iteration: {num_randomize * num_best * samps_per_seed}')
```

```
Total numbers searched per iteration: 100
```

```
# fit the model
```

```
trainer.fit(model, train_loader)
```

```
trainer.validate(model, train_loader)
```

```
trainer.save_checkpoint(os.path.join(out_path, f'{gen}_model.ckpt'))
```

```
clear_output()
```

```
collector.to_csv(os.path.join(out_path, 'all_resultsRL.csv'), index=False)
```

Reinforcement learning

```
# define parameters
model_path      = path+'/version_0/final_model.ckpt'
out_path        = 'RESULTS_retrain_{string_type}_PI'
num_workers     = 6

# control sampling
# total number of samples = num_best * samps_per_seed * num_randomize
num_generations = 15
num_best        = 10          # number of top molecules to generate seeds
num_randomize   = 5          # number of randomized seeds
samps_per_seed  = 2          # number of samples for each seed
num_seed_chars  = None       # number of initial characters to sample from
                                # if none, sample characters 1/4 - 3/4 of strings
temperature     = 1.1        # < 1.0 is less random, > 1.0 is more random
retrain         = True       # retrain the network on new molecules

print(f'Total numbers searched per iteration: {num_randomize * num_best * samps_per_seed}')
```

} → Number of new structures generated per generation

→ Control the randomness of the generating process

Total numbers searched per iteration: 100 → Total number of structures generated in each generation

```
# fit the model
trainer.fit(model, train_loader)
trainer.validate(model, train_loader)
trainer.save_checkpoint(os.path.join(out_path, f'{gen}_model.ckpt'))

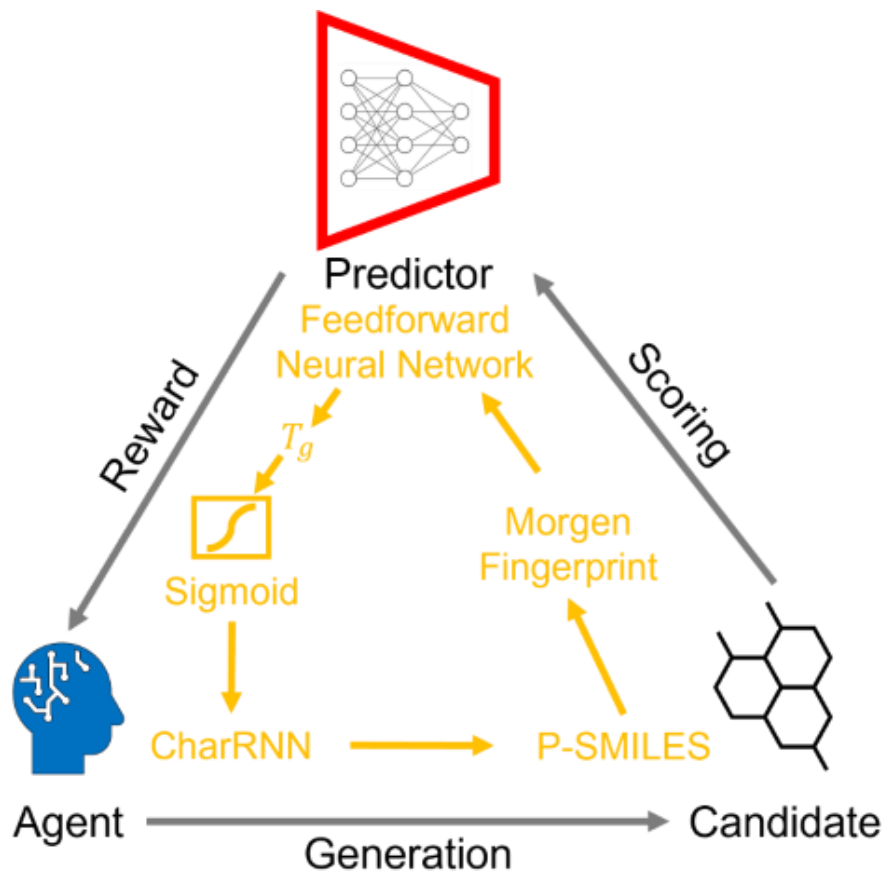
clear_output()

collector.to_csv(os.path.join(out_path, 'all_resultsRL.csv'), index=False)
```

→ Store the model resulting from reinforcement learning

→ Save the generated results at each generation

Reinforcement learning



```

# create index for results
if not os.path.isdir(out_path):
    os.mkdir(out_path)

## starting algorithm
# step 1: gather data, get the top molecules
start_dfl = pd.DataFrame({'smiles': mol_list}) # smiles are canonicalized, no duplicates
start_of['fitness'] = start_of['smiles'].apply(fitness_function)
top_n = start_of.nlargest(top_n, 'fitness')
original_best = top_n.iloc[0]['fitness']

# load model and collector
model = language_model.load_from_checkpoint(model_path)
collector = pl.data_loader(columns = ['smiles', 'fitness', 'generation'])
best_collector = pl.data_loader(columns = ['smiles', 'fitness', 'generation'])

for gen in range(num_generations):

    # load previous model
    if gen > 0:
        model = language_model.load_from_checkpoint(os.path.join(out_path, f'gen-{gen}.ckpt'))

    print(f'generation {gen}')
    cika = ['smiles', 'smiles'] if string_type == 'smiles' else ['smiles']
    new_dfl = pl.data_loader(columns=cika)

    # Step 1: create seeds
    rand_string = []
    for k, k_n in top_n.iterrows(): # loop through top molecules
        if string_type == 'smiles':
            # randomize the smiles before selecting to smiles to increase diversity
            # make sure the new smiles can be encoded by the automata
            rand_sfx = []
            while len(rand_sfx) < max_randomize:
                sfx_list = split_randomize_smiles(rand_sfx['smiles'], max_randomize)
                sfx_list = [sfx.encode(x) for x in sfx_list]
                try:
                    # = [m.encode_string(s) for s in sfx_list]
                    rand_sfx.extend(sfx_list)
                except:
                    print('error with randomized SMILES encoding.')
            rand_string.extend(rand_sfx)
        else:
            rand_string.extend([rand_sfx['smiles']] * max_randomize)

    for k, k_n in enumerate(rand_string):

        # Step 2: select starting seeds from top molecules for sampling
        if max_seed_size is None:
            frac = np.random.rand() / 2.0 + 0.5 # from 0.5 to 0.5
            len_s = len(new_smiles) if string_type == 'smiles' else len(
                start_of['smiles', k_n].iloc[0] * frac, string_type)
            seed = start_of['smiles', k_n].iloc[0: len_s, string_type]
            new_dfl.extend(seed)
        else:
            seed = start_of['smiles', k_n].iloc[0: max_seed_size, string_type]
            new_dfl.extend(seed)

    # Step 3: sample from seeds, RNN duplicates
    new_smpls = list() # sample_gen, seed, len(new_dfl)
    while new_smpls < n:
        sampled_molecules = model.collect(seed, new_dfl, sample_size, temperature = temperature)
        if string_type == 'smiles':
            new_smpls, new_smiles = de_logits_to_smiles(sampled_molecules, return_smiles=True)
        else:
            new_dfl.extend(new_smiles)
            new_smiles = new_smiles['smiles']
            new_dfl = pl.data_loader(columns=new_smiles)
            new_dfl = pl.minimize(new_dfl, new)
            new_dfl_duplicate = new_dfl['smiles']
            new_dfl_duplicate = list(new_dfl_duplicate)
            new_smpls = new_dfl_duplicate + len(new_dfl)
            # import pdb; pdb.set_trace()

    new_smiles = de_logits_to_smiles(sampled_molecules, canonicalize=False)
    new_dfl = pl.data_loader(columns=new_smiles)
    new_dfl = pl.minimize(new_dfl, new)
    # new_dfl_duplicate = new_dfl['smiles']
    # new_dfl_duplicate = list(new_dfl_duplicate)
    # new_smpls = new_dfl_duplicate + len(new_dfl)

    # Step 4: calculate new fitnesses and gather results
    new_dfl['fitness'] = new_dfl['smiles'].apply(fitness_function)
    new_dfl['generation'] = [gen] * len(new_dfl)
    collector = pl.data_loader(columns=new_dfl, ignore_index=True)
    best_collector['fitness'] = collector['fitness'].argmax(axis=0)
    best_in_gen = new_dfl.nlargest(1, 'fitness')
    best_overall = collector.nlargest(1, 'fitness')
    best_overall['generation'] = gen
    best_collector = pl.data_loader(columns=best_collector, best_overall, ignore_index=True)

    print(f'best in generation: {best_in_gen['smiles'].iloc[0]} {best_in_gen['fitness'].iloc[0]}')
    print(f'best overall: {best_overall['smiles'].iloc[0]} {best_overall['fitness'].iloc[0]}')

    train = collector.nlargest(top_n, 'fitness')
    top_n = top_n.reset_index(drop = True)

    # Step 5: train the network on the new molecules if desired
    if train:
        # create new dataset depending on string type
        train_strings = new_dfl['fitness', string_type].tolist() # remove invalid molecules before training
        if string_type == 'smiles':
            dataset = SeqDataset(train_strings, de_vocab, de_len_molecule)
        else:
            dataset = MoleculeDataset(train_strings, de_vocab, de_len_molecule)
        train_loader = torch.utils.data.DataLoader(dataset, batch_size = de_batch_size)

    # set model parameters
    model.verbose = True # use a smaller learning rate for fine-tuning
    model.learning_rate = lr * 0.1

    if torch.cuda_is_available():
        trainer = pl.Trainer(
            accelerator = 'gpu',
            devices = 1,
            check_val_every_n_epoch = 1,
            logger=False,
            max_epochs=1,
            num_sanity_val_steps = 0)
    else:
        trainer = pl.Trainer(
            accelerator = 'cpu',
            num_sanity_val_steps = 0,
            check_val_every_n_epoch = 1,
            logger=False,
            max_epochs=1,
            num_sanity_val_steps = 0)

    # fit the model
    trainer.fit(model, train_loader)
    trainer.save_checkpoint(os.path.join(out_path, f'gen{gen}.ckpt'))

    close_script()

```

Reinforcement learning

```
def is_valid_smiles(smiles):  
    try:  
        mol = Chem.MolFromSmiles(smiles)  
        return mol is not None  
    except:  
        return False
```

→ Validate the molecular structure from generated SMILES strings

```
mask = collector["smiles"].str.count('\*') == 2  
collector_valid = collector[mask]  
from IPython.display import clear_output  
valid_indices = []
```

→ Check whether the results are linear polymers

```
for index, row in collector_valid.iterrows():  
    if is_valid_smiles(row['smiles']):  
        valid_indices.append(index)  
    clear_output()
```

```
collector_valid = collector_valid.loc[valid_indices]  
collector_valid_unique = collector_valid.drop_duplicates(subset='smiles')  
collector_valid_unique
```

→ Remove duplicate SMILES strings from the results

	smiles	fitness	generation
1	*c1ccc(-c2nc3cc(Oc4ccc(NC(=O)c5cc(C(=O)Nc6ccc(...	106.189568	0
12	*c1ccc(-c2nc3cc(Oc4ccc(NC(=O)c5cc(C(=O)Nc6ccc(...	108.594574	0
20	*C(=O)Nc1ccc(-c2ccc(NC(=O)c3ccc4c(c3)C(=O)N(c3...	90.608231	0
21	*C(=O)Nc1ccc(-c2ccc(NC(=O)c3ccc4c(c3)C(=O)N(c3...	102.784721	0
22	*C(=O)Nc1ccc(-c2ccc(NC(=O)c3ccc4c(c3)C(=O)N(c3...	101.930962	0
...	...	...	...
1398	*c1ccc(-c2nc3cc(Oc4ccc(NC(=O)c5cc(C(=O)Nc6ccc(...	119.476013	13
1443	*c1ccc(-c2nc3cc(Oc4ccc(NC(=O)c5cc(C(=O)Nc6ccc(...	130.227295	14
1459	*c1ccc(-c2nc3cc(Oc4ccc(NC(=O)c5cc(C(=O)Nc6ccc(...	130.677887	14
1476	*c1ccc(-c2nc3cc(Oc4ccc(NC(=O)c5cc(C(=O)Nc6ccc(...	125.220497	14
1481	*c1ccc(-c2nc3cc(Oc4ccc(NC(=O)c5cc(C(=O)Nc6ccc(...	114.343025	14

The number of valid structures is limited because the number of structures generated at each generation was set to a low value.

Only 14 generations were trained as a demonstration, due to computational resource limitations

Reinforcement learning

```
fn = collector_valid_unique.smiles.apply(Chem.MolFromSmiles).apply(lambda m: AllChem.GetMorganFingerprintAsBitVect(m, radius=3, nBits=1024))
fn_list = fn.tolist()
new_inputs = torch.tensor(fn_list, dtype=torch.float32)
```

```
predictions = loaded_model(new_inputs)
predictions = predictions.detach().numpy()
collector_valid_unique['Tg'] = predictions
collector['Tg'] = None
collector = collector.sort_values(by='generation')
smiles_to_tg = pd.Series(collector_valid_unique['Tg'].values, index=collector_valid_unique['smiles']).to_dict()
collector['Tg'] = collector['smiles'].map(smiles_to_tg)
```

Predict the Tg of the generated results for each generation

```
# Initialize data structures to store results
```

```
validity = []
redundancy = []
average_Tg = []
```

```
# Iterate over each generation to perform calculations
```

```
generations = collector['generation'].unique()
```

```
for gen in generations:
```

```
    gen_df = collector[collector['generation'] == gen]
```

```
    # Validity calculation: proportion of non-NaN Tg values out of the total
```

```
    valid_samples = gen_df['Tg'].notna()
    validity_rate = valid_samples.sum() / len(gen_df)
    validity.append(validity_rate)
```

```
    # Redundancy calculation: proportion of unique SMILES out of the total
```

```
    unique_smiles = gen_df['smiles'].nunique()
    redundancy_rate = 1 - unique_smiles / len(gen_df)
    redundancy.append(redundancy_rate)
```

```
    # Average Tg value of valid samples
```

```
    avg_Tg = gen_df[valid_samples]['Tg'].mean()
    average_Tg.append(avg_Tg)
```

Statistics of validity, duplication rate, and average prediction results for each generation

Reinforcement learning

```
import matplotlib.pyplot as plt
# Plotting
fig, ax1 = plt.subplots(figsize=(10, 6))

# Plotting Average Tg on the left y-axis
ax1.plot(generations, average_Tg, label='Average Tg of Valid Samples', color='g')
ax1.set_xlabel('Iteration Steps')
ax1.set_ylabel('Average Tg Value', color='g')
ax1.tick_params(axis='y', labelcolor='g')
ax1.set_title('Effect of Iteration Steps on Validity, Redundancy, and Average Tg')

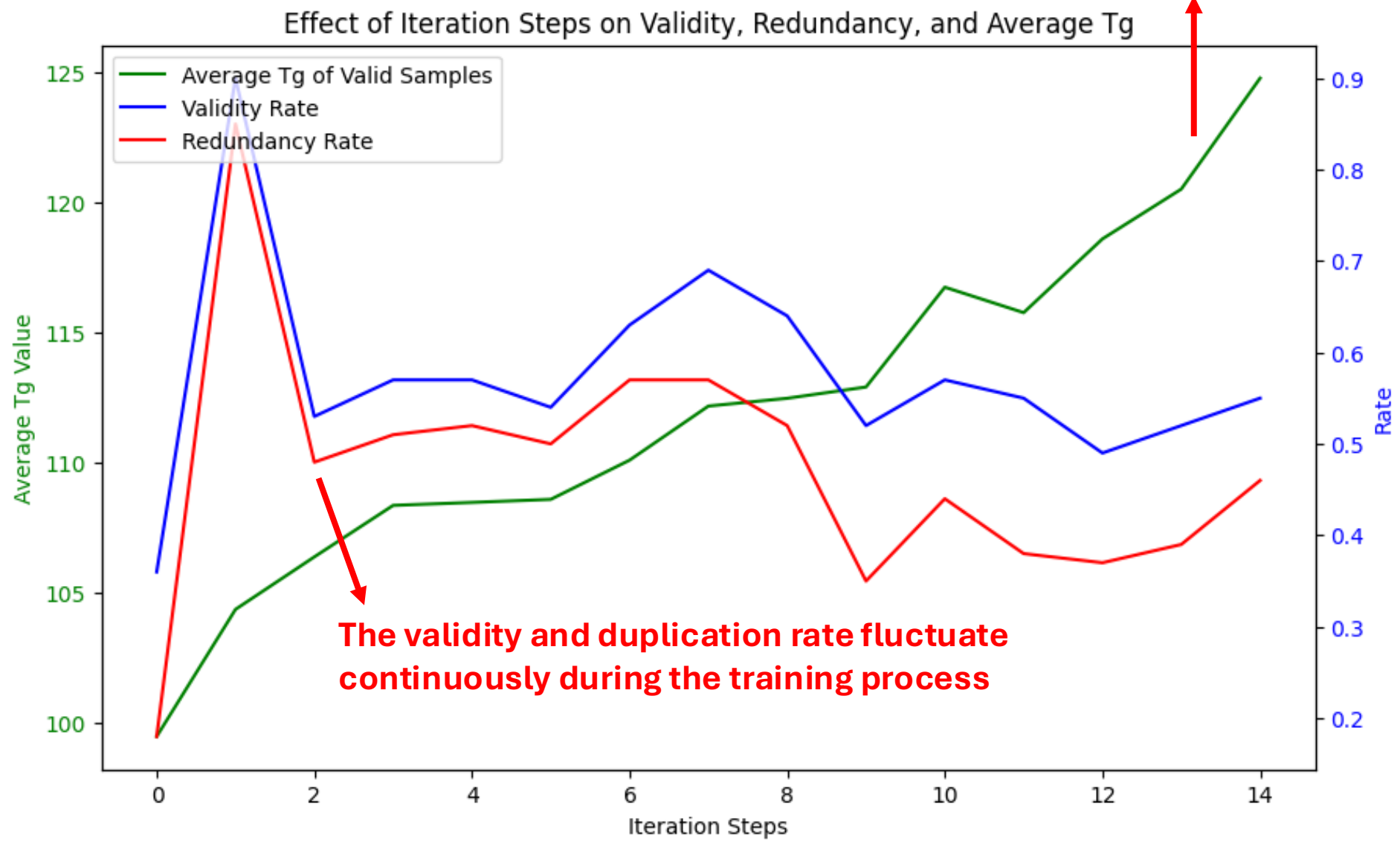
# Using a second y-axis for Validity and Redundancy rates
ax2 = ax1.twinx()
ax2.plot(generations, validity, label='Validity Rate', color='b')
ax2.plot(generations, redundancy, label='Redundancy Rate', color='r')
ax2.set_ylabel('Rate', color='b')
ax2.tick_params(axis='y', labelcolor='b')

# Adding a legend
lines, labels = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax2.legend(lines + lines2, labels + labels2, loc='upper left')
```

Plot the validity, duplication rate, and average prediction results for each generation

Reinforcement learning

The average Tg of the generated results gradually increases with training generations



T-SNE plot of the results obtained from reinforcement learning

```
from sklearn.manifold import TSNE
import time
```

```
collector.dropna(subset=['Tg'], inplace=True)
```

```
Trainset = pd.read_csv(path+'PolyInfo.csv') → Read in the training set
```

```
combined_data = pd.concat([collector['smiles'], Trainset['smiles']], ignore_index=True) → Combine all the polymer structures
```

```
fn = pd.DataFrame(combined_data)['smiles'].apply(Chem.MolFromSmiles).dropna().apply(lambda m: AllChem.GetMorganFingerprintAsBitVect(m, radius=3, nBits=2048)) → Generate Morgan Fingerprint
```

```
X_fingerprint_All_Polyinfo = np.array(fn.tolist())
```

```
time_start = time.time()
```

```
tsne = TSNE(n_components=2, verbose=1, perplexity=40, n_iter=300, random_state=100) → TSNE analysis
```

```
tsne_results = tsne.fit_transform(X_fingerprint_All_Polyinfo)
```

```
collector['TSNE 1'] = tsne_results[:collector.shape[0], 0]
```

```
collector['TSNE 2'] = tsne_results[:collector.shape[0], 1]
```

```
Trainset['TSNE 1'] = tsne_results[collector.shape[0]:, 0]
```

```
Trainset['TSNE 2'] = tsne_results[collector.shape[0]:, 1]
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
x_reinvent = collector['TSNE 1']
```

```
y_reinvent = collector['TSNE 2']
```

```
x_trainset = Trainset['TSNE 1']
```

```
y_trainset = Trainset['TSNE 2']
```

```
generation = collector['generation'] → Plot the distribution of the training set using gray points
```

```
alpha = 0.2
```

```
fig = plt.figure(figsize=(10, 8))
```

```
plt.scatter(x_trainset, y_trainset, c='gray', s=30, alpha=1, marker='o', label='Trainset')
```

```
plt.scatter(x_reinvent, y_reinvent, c=generation, cmap='viridis', s=45, alpha=alpha, marker='o', label='REINVENT') → Plot the generated structures of each generation, using color to indicate generation changes
```

```
plt.colorbar(label='Generation')
```

```
plt.xlabel('TSNE 1', fontsize=24)
```

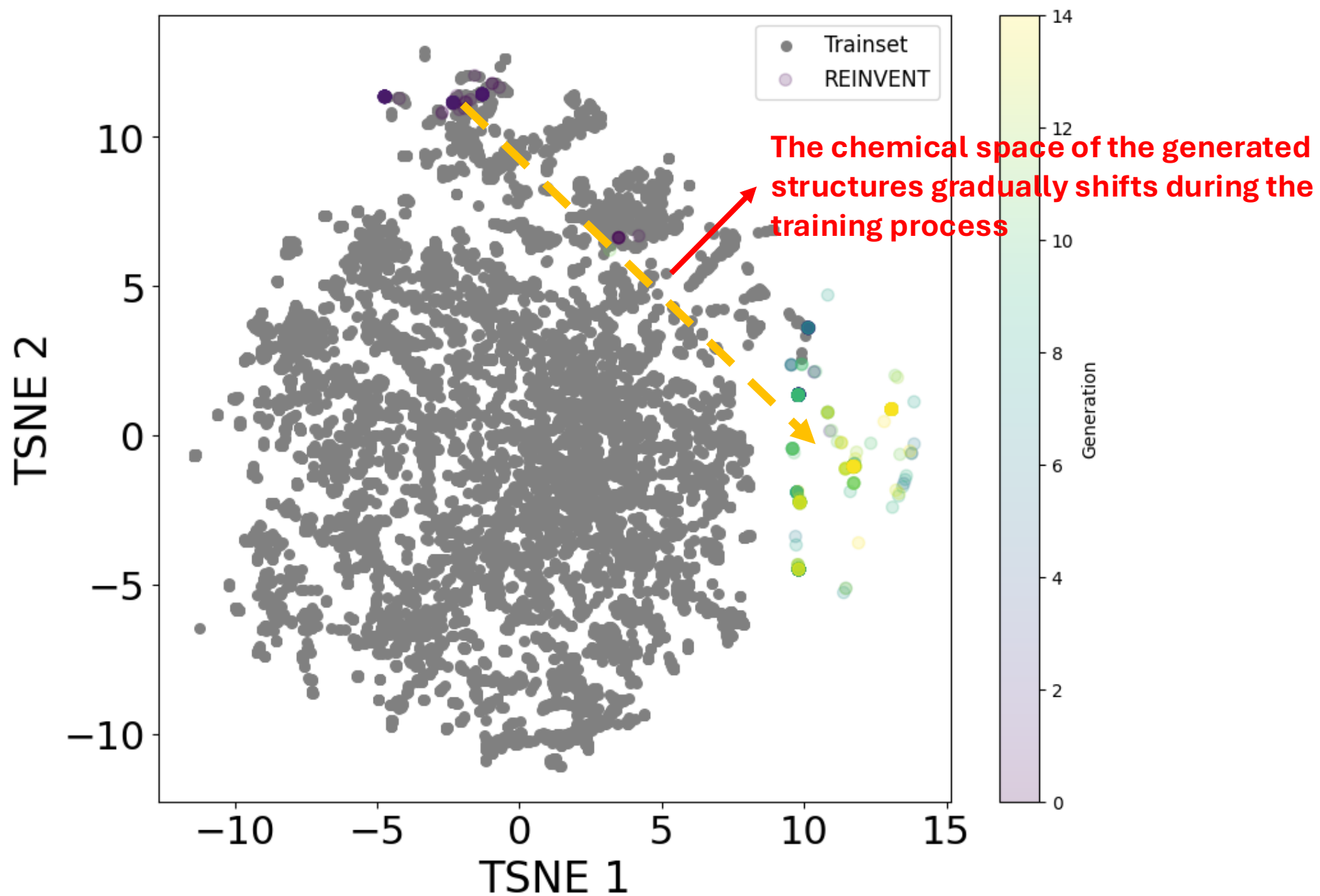
```
plt.ylabel('TSNE 2', fontsize=24)
```

```
plt.legend(fontsize=12)
```

```
plt.tick_params(axis='both', labelsize=22)
```

```
plt.show()
```

T-SNE plot of the results obtained from reinforcement learning



Generate hypothetical polymer structures using the trained CharRNN

```
# define parameters
model_path      = f'/version_0/final_model.ckpt'
out_path        = f'RESULTS_retrain'
num_workers     = 6

# control sampling
# total number of samples = num_best * samps_per_seed * num_randomize
num_generations = 14
num_best        = 10          # number of top molecules to generate seeds
num_randomize   = 5           # number of randomized smiles
samps_per_seed  = 10          # number of samples for each seed
num_seed_chars  = None        # number of initial characters to sample from
                                # if none, sample characters 1/4 - 3/4 of strings
temperature     = 1.1         # < 1.0 is less random, > 1.0 is more random
retrain         = True        # retrain the network on new molecules

print(f'Total numbers searched per iteration: {num_randomize * num_best * samps_per_seed}')

# Step 3: Sample from seeds, KEEP duplicates
num_samps = (i+1) * samps_per_seed - len(new_df)
while num_samps > 0:
    sampled_molecules = model.sample(onehot_seed, num_samps, temperature = temperature)
    if string_type == 'selfies':
        new_smiles, new_selfies = dm.logits_to_smiles(sampled_molecules, return_selfies=True)
        new = pd.DataFrame({'smiles': new_smiles, 'selfies': new_selfies})
    else:
        new_smiles = dm.logits_to_smiles(sampled_molecules, canonicalize=False)
        new = pd.DataFrame({'smiles': new_smiles})
    new_df = pd.concat([new_df, new])
    num_samps = (i+1) * samps_per_seed - len(new_df)
    # import pdb; pdb.set_trace()

new_df.to_csv(os.path.join(out_path, 'generate_RL.csv'), index=False)
```

Path of the model

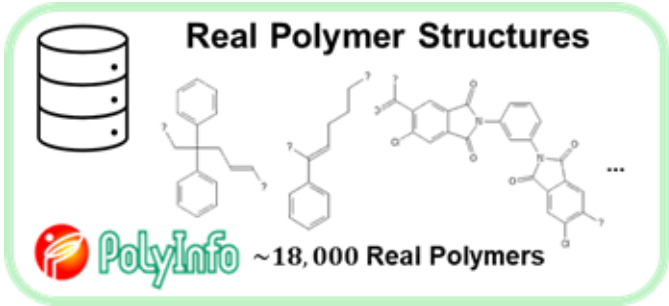
Number of generated structures

Control the randomness of the generation process

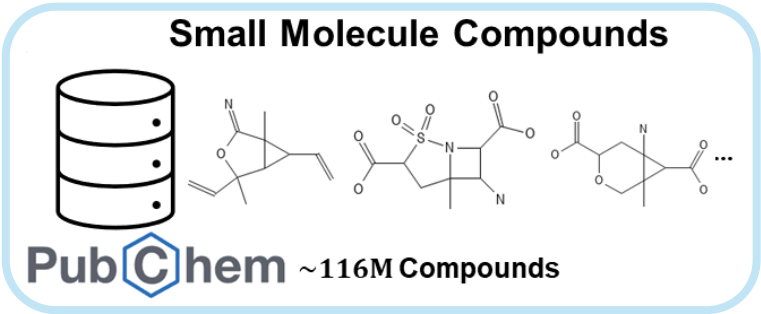
Append the generated results to the DataFrame

Save the generated results

Data shortage



V.S.



Property

Count

Log-transformed octanol/water partition coefficient ($\log P$)

14,207

Blood-brain barrier (BBB) permeability

7,807



Property

Count

Glass transition temperature

6,906

Melting temperature

3,633

Decomposition temperature

5,237

Thermal conductivity

68

Heat capacity

79

Property

Count

Young's modulus

923

Yield strength

230

Break strength

982

Dielectric constant

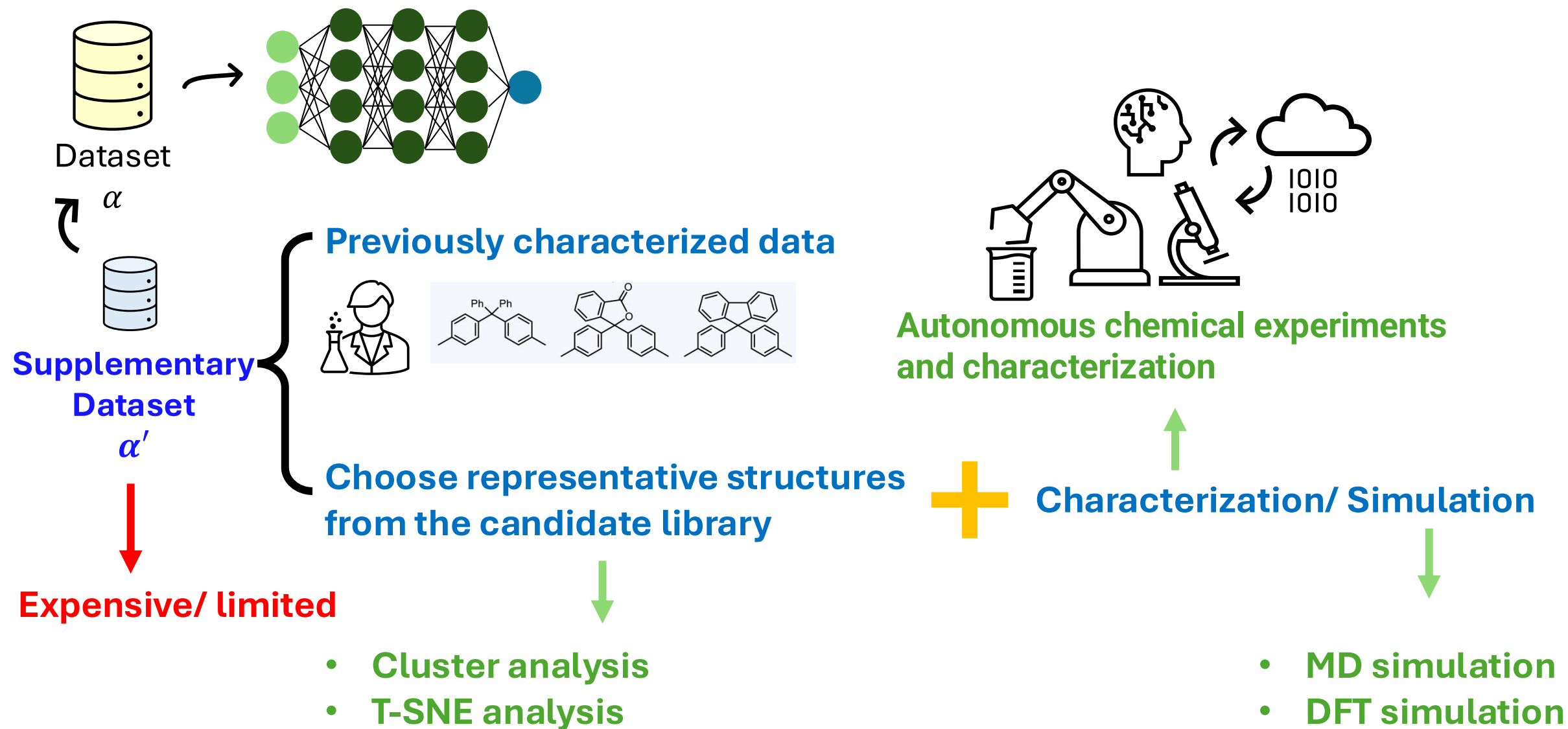
627

Bandgap

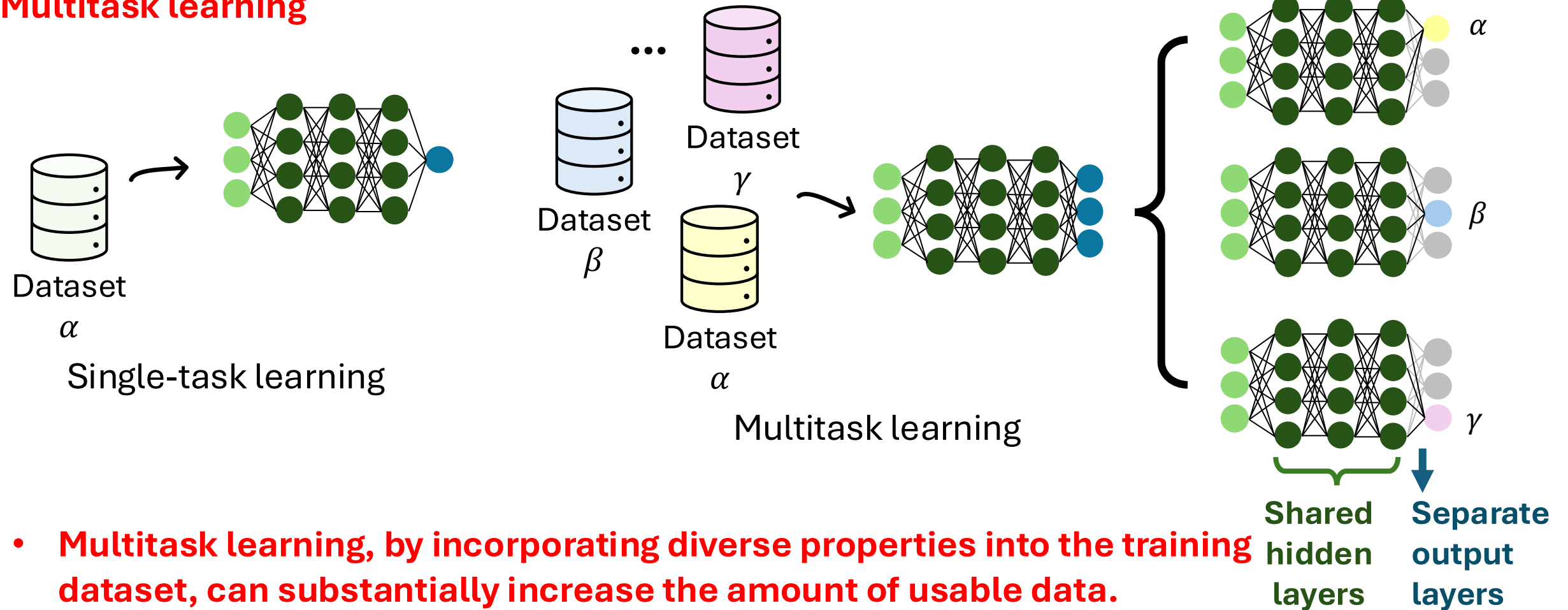
3,379

Training Prediction Models with Limited Data

Limited data coverage



Multitask learning



Training Prediction Models with Limited Data

Multitask learning

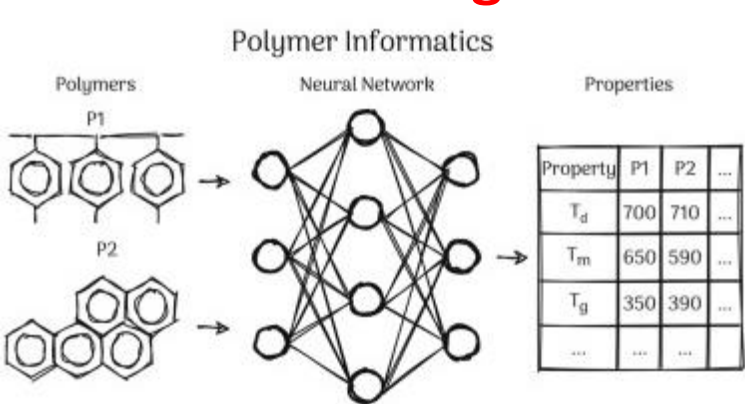
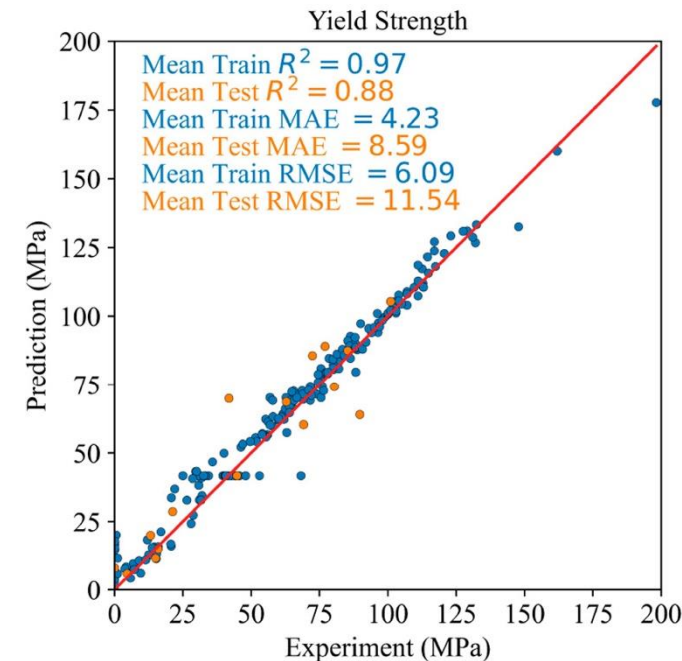
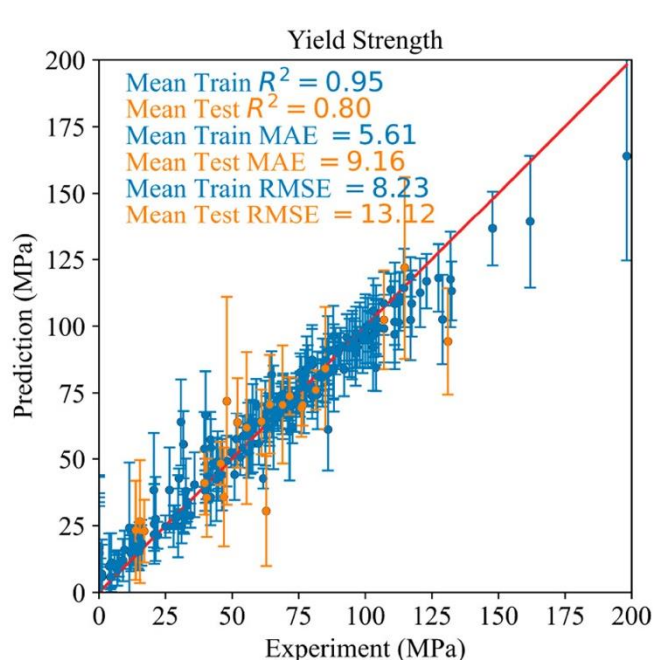
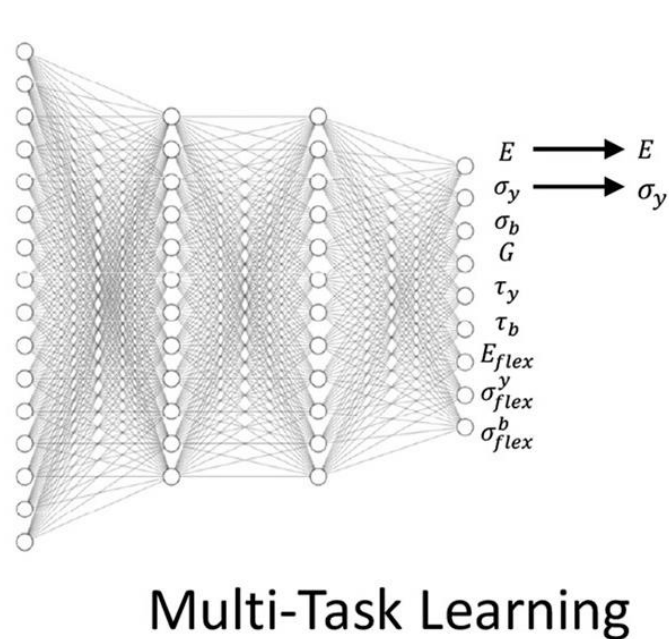


Table 2. Averages of the normalized root mean squared errors values

Model	All	Categories					
		Thermal	Thermod. & physical	Electronic	Optical & dielectric	Mech.	Solubility & permeability
GP-ST	0.93	0.92	0.86	0.88	0.98	1.00	0.93
NN-ST	0.89	0.95	0.76	0.91	0.91	0.95	0.94
NN-MT1	0.89	0.98	0.83	0.97	0.83	0.94	0.92
NN-MT2-all	0.79	0.97	0.82	0.96	0.69	0.89	0.70
NN-MT2-sub	0.65	0.94	0.87	0.79	0.47	0.82	0.46

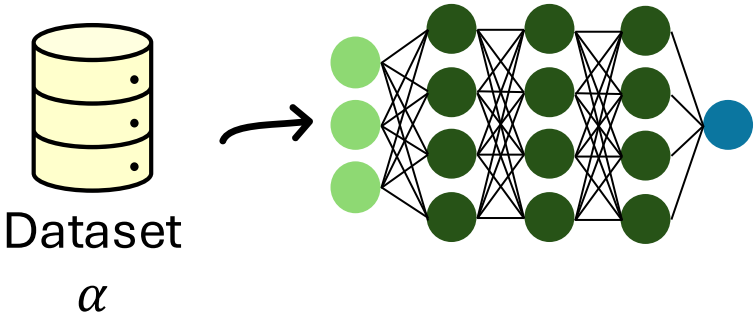
The root mean squared error averages were normalized for each property so that the maximal value is 1.

Kuenneth, C., Rajan, A.C., Tran, H., Chen, L., Kim, C. and Ramprasad, R., 2021. Polymer informatics with multi-task learning. *Patterns*, 2(4).

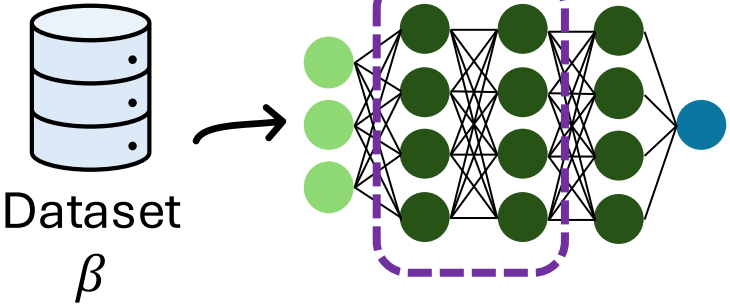


Yue, T., He, J., Tao, L. and Li, Y., 2023. High-throughput screening and prediction of high modulus of resilience polymers using explainable machine learning. *Journal of Chemical Theory and Computation*, 19(14), pp.4641-4653.

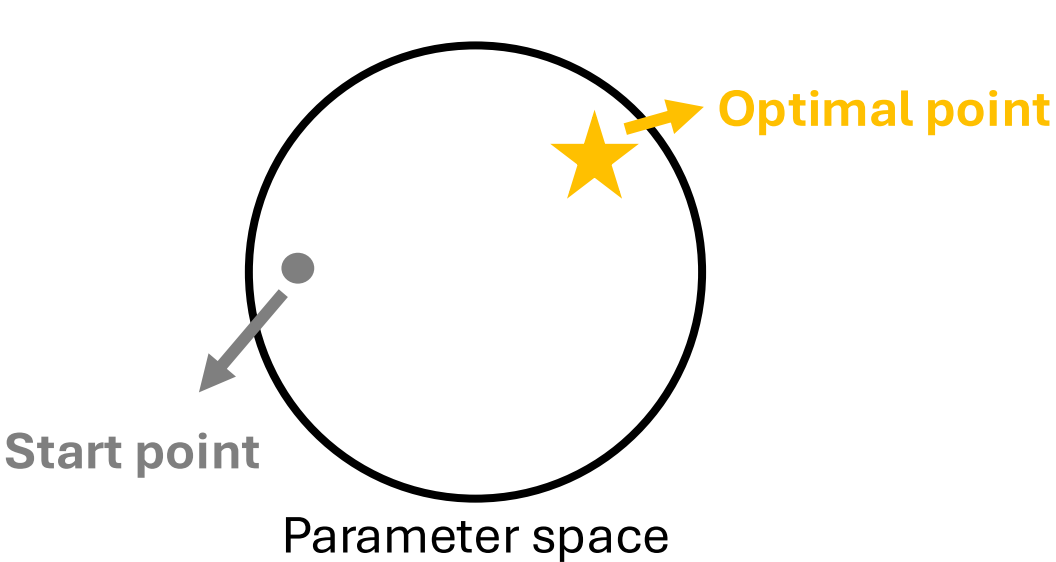
Transfer Learning



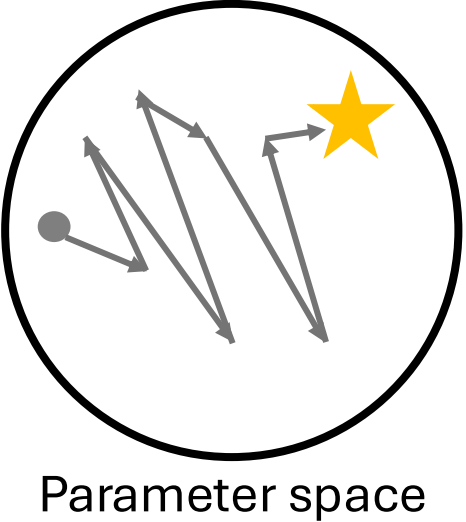
Pre-training



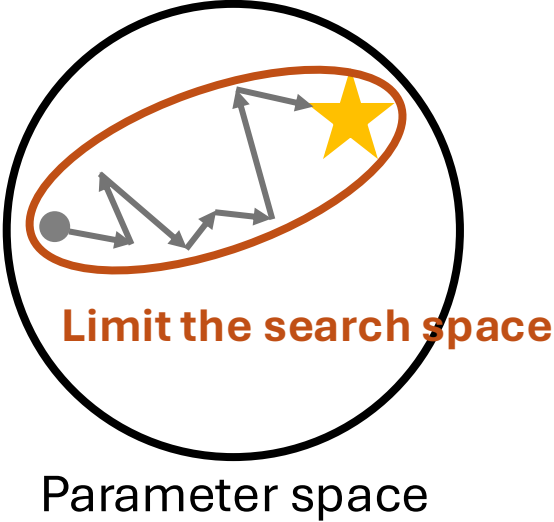
Fine-tuning



Direct learning

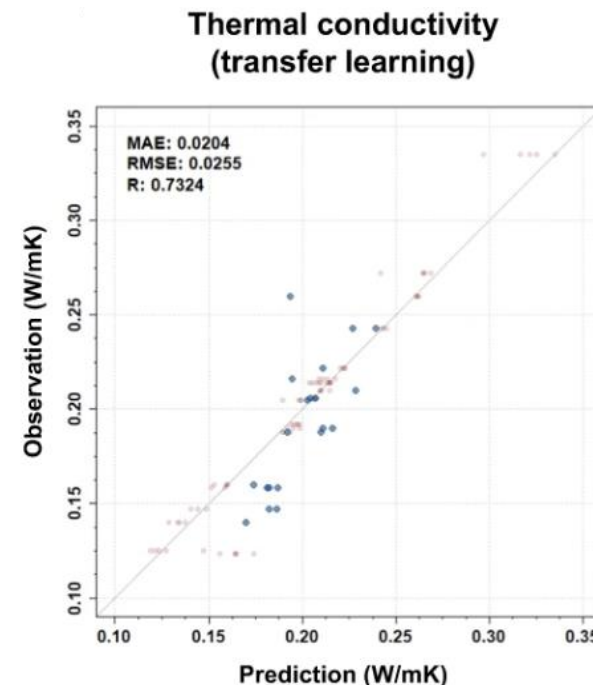
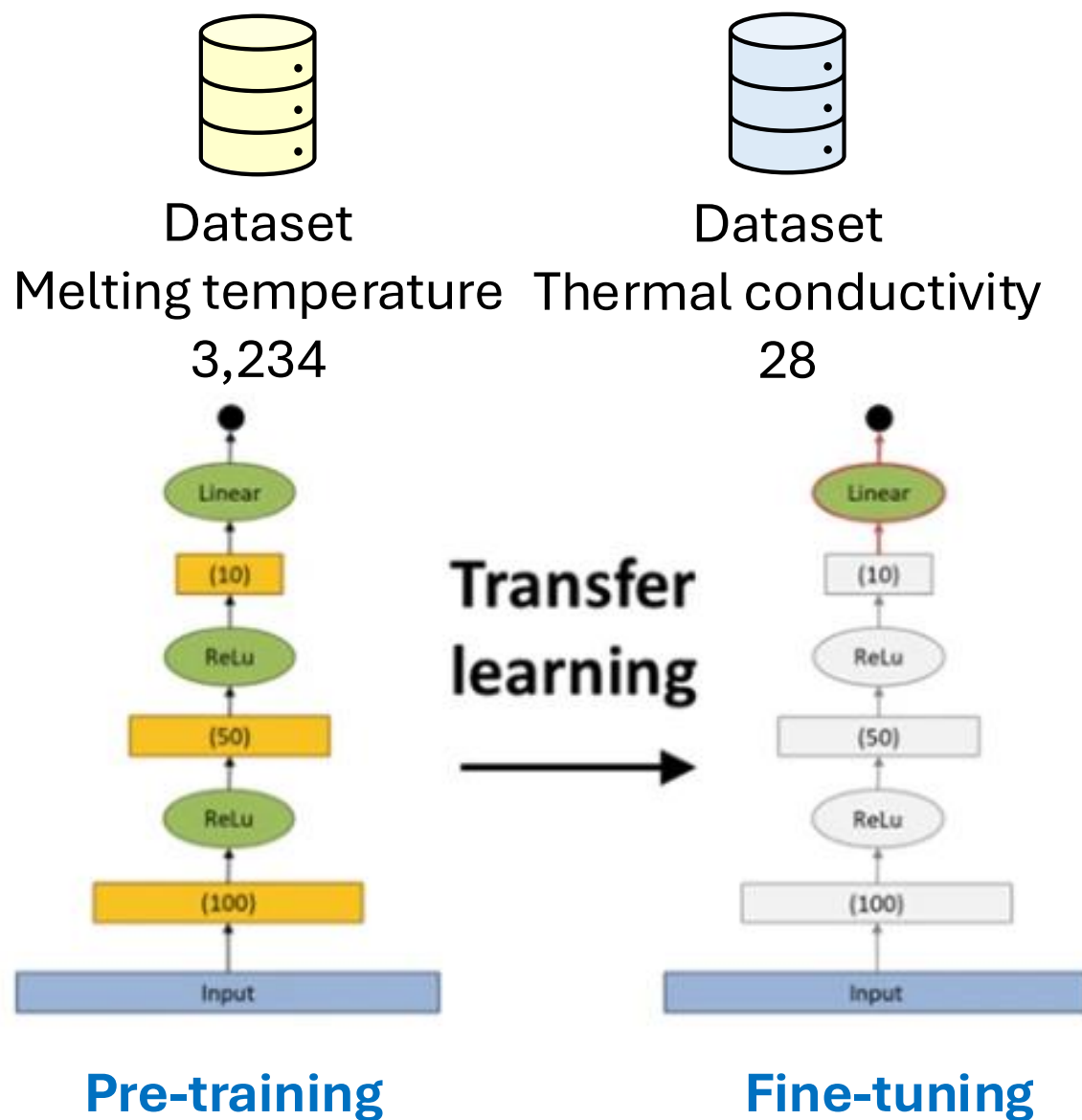


Transfer Learning

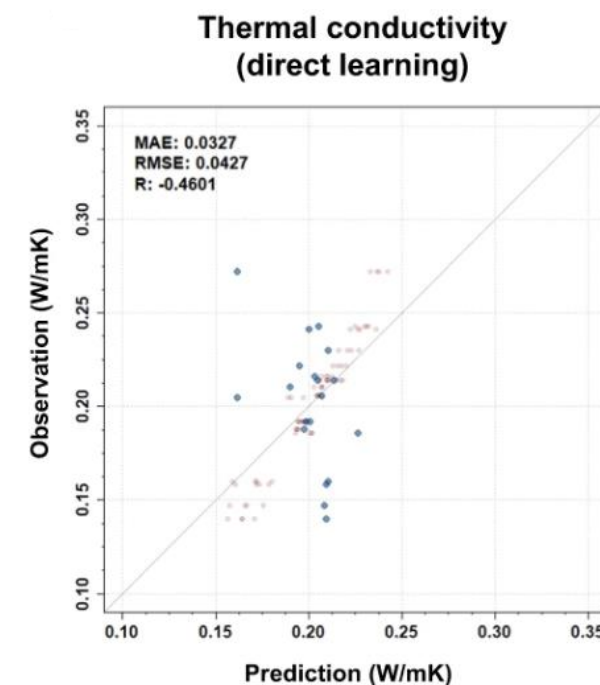


Training Prediction Models with Limited Data

Transfer Learning



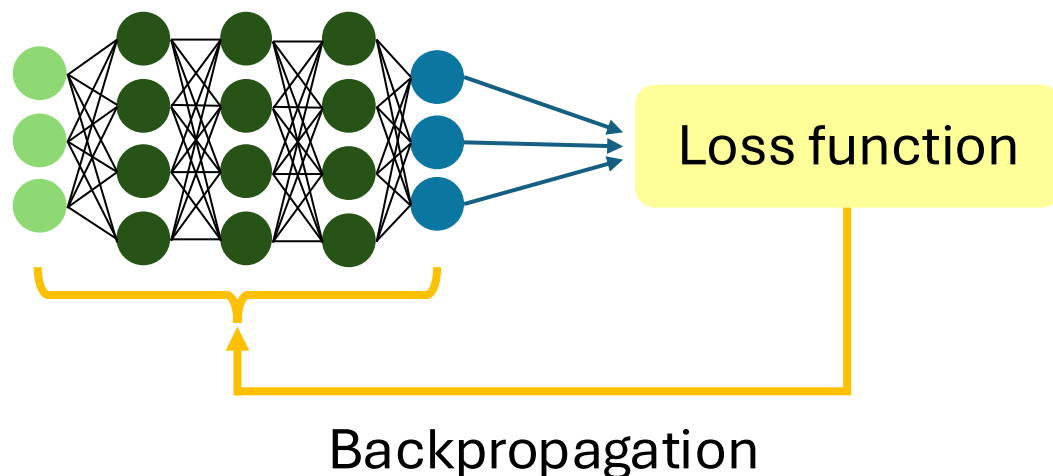
$$R^2: 0.73$$



$$R^2: -0.46$$

Wu, S., Kondo, Y., Kakimoto, M.A., Yang, B., Yamada, H., Kuwajima, I., Lambard, G., Hongo, K., Xu, Y., Shiomi, J. and Schick, C., 2019. Machine-learning-assisted discovery of polymers with high thermal conductivity using a molecular design algorithm. *Npj Computational Materials*, 5(1), p.66.

Physics-informed Neural Networks



Traditional NN: Data loss

$$\mathcal{L} = (A - \hat{A})^2 + (B - \hat{B})^2 + (C - \hat{C})^2$$

PINN: Data loss + Physics loss

$$\mathcal{L}_1 = (A - \hat{A})^2 + (B - \hat{B})^2 + (C - \hat{C})^2$$

$$\mathcal{L}_2 = f(\hat{A}, \hat{B}, \hat{C})$$

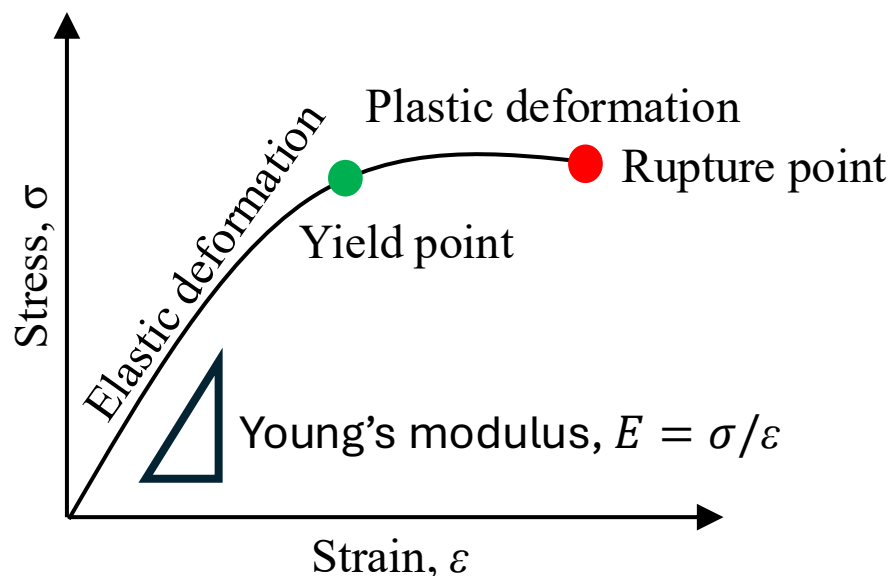
$$\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_2$$

Only based on prediction values

A, B, C label values

$\hat{A}, \hat{B}, \hat{C}$ predicted values

$f(\cdot)$ enforce equations/ governing equations



$$\mathcal{L}_1 = (\hat{E} - E)^2 + (\hat{\sigma} - \sigma)^2 + (\hat{\varepsilon} - \varepsilon)^2$$

$$\mathcal{L}_2 = \hat{E} - \hat{\sigma} / \hat{\varepsilon}$$

$$\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_2$$

E, σ, ε label values

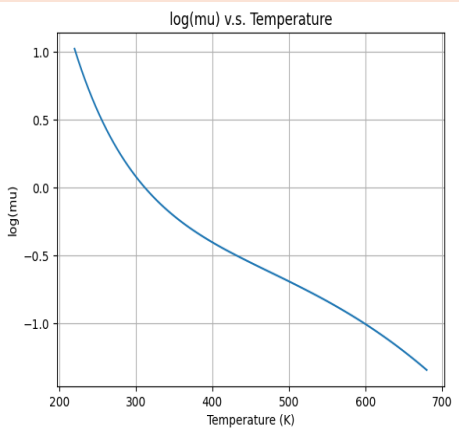
$\hat{E}, \hat{\sigma}, \hat{\varepsilon}$ predicted values

Training Prediction Models with Limited Data

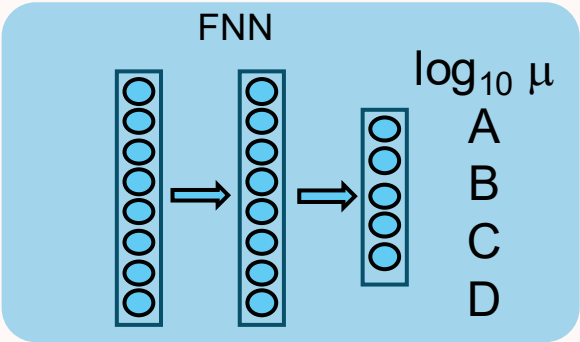
Physics-informed Neural Networks

Transport Properties of Chemicals and Hydrocarbons

$$\log_{10} \mu = A + B/T + CT + DT^2$$



6,120 unique compounds



Loss function

$$\mathcal{L}_1 = (A - \hat{A})^2 + (B - \hat{B})^2 + (C - \hat{C})^2 + (D - \hat{D})^2$$

$$\mathcal{L}_2 = [\log(\mu)' - \log(\mu)]^2$$

$$\mathcal{L}_3 = \left[\log_{10} \mu - \left(\hat{A} + \frac{\hat{B}}{T} + \hat{C}T + \hat{D}T^2 \right) \right]^2$$

$$\mathcal{L} = \mathcal{L}_1 + \mathcal{L}_2 + \lambda \mathcal{L}_3$$

Data loss

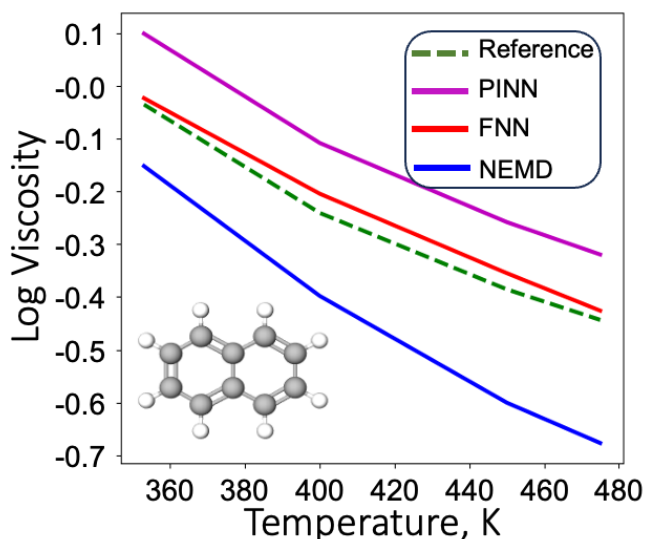
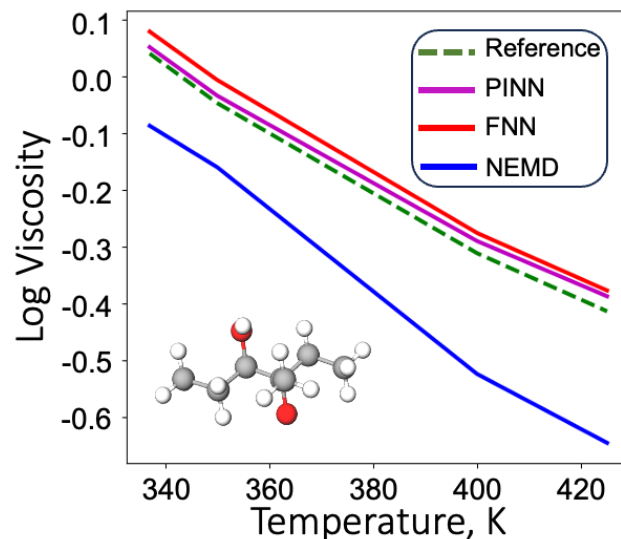
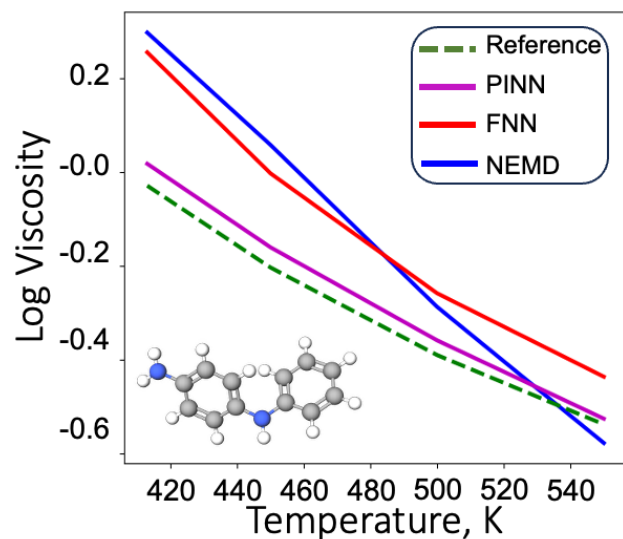
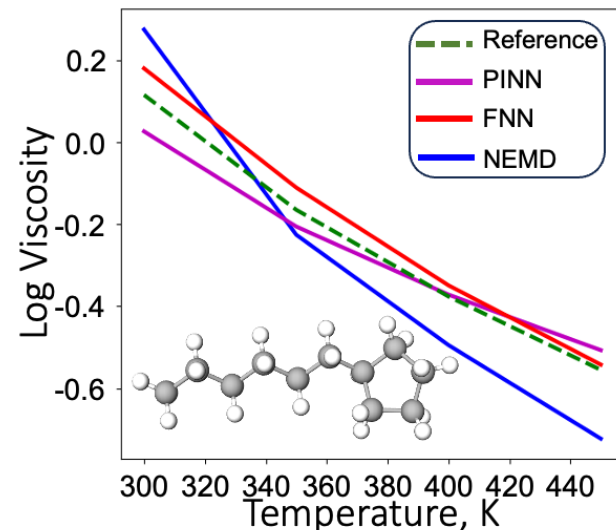
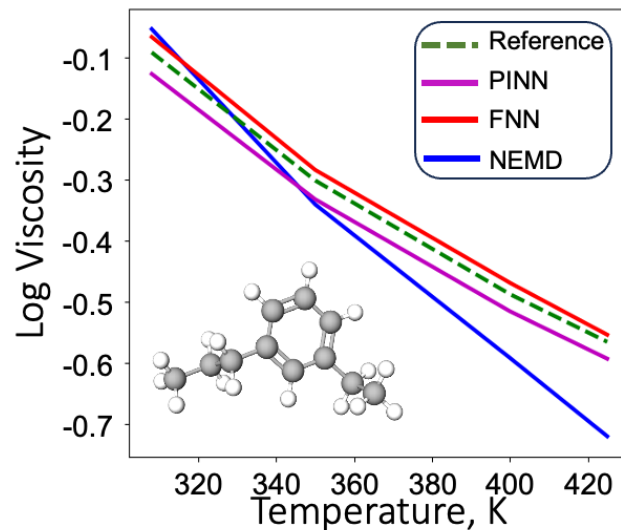
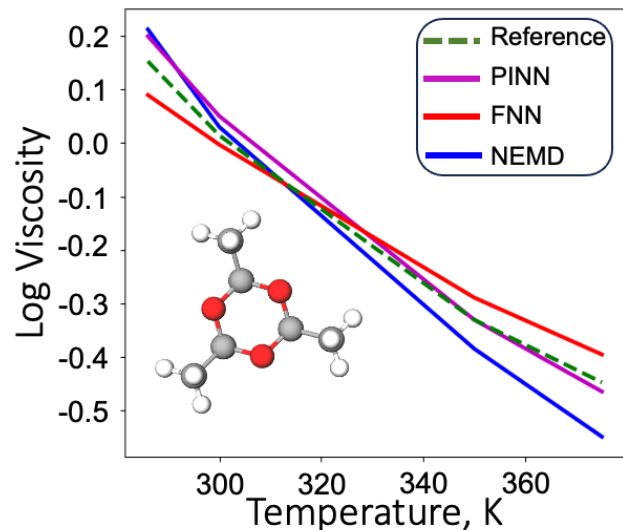
Physics loss

λ	R^2 (pred log μ)	R^2 (cal log μ)	R^2 (pred A)	R^2 (pred B)	R^2 (pred C)	R^2 (pred D)
0.00001	0.99	0.91	0.97	0.98	0.97	0.97
0.001	0.98	0.90	0.95	0.96	0.95	0.95
0.01	0.98	0.91	0.95	0.96	0.95	0.95
0.1	0.99	0.97	0.96	0.96	0.96	0.95
1	0.96	0.95	0.77	0.84	0.71	0.68
-	-	0.63	0.75	0.82	0.67	0.71

Only data loss

Training Prediction Models with Limited Data

Physics-informed Neural Networks



- **Temperature-dependency can be captured so well.**
- **Physics loss also contribute to parameter optimization.**

Question Time

Thank you!

Tianle Yue (tyue4@wisc.edu)